

Project: Football Players Valuation Analysis

Phase 1: Data Ingestion & SQL Integration

This initial stage establishes the project's foundation by connecting to external data sources (Kaggle), configuring the environment, and ensuring raw data persistence in a PostgreSQL database for SQL-based processing.

```
In [1]: import os

# 1. Load credentials from your .env file first
def load_kaggle_credentials(file_path):
    with open(file_path, 'r') as f:
        # Expecting just the API Key string inside the file
        token = f.read().strip()
    return token

api_token = load_kaggle_credentials('Kaggle_token.env')

# 2. Set environment variables BEFORE importing kaggle
os.environ['KAGGLE_KEY'] = api_token
os.environ['KAGGLE_USERNAME'] = "guilhermeoyakawa"

# 3. Now import kaggle and authenticate
import kaggle
from kaggle.api.kaggle_api_extended import KaggleApi

api = KaggleApi()
api.authenticate()

print("Kaggle Authentication Successful!")
```

Kaggle Authentication Successful!

```
In [2]: # Standard Libraries for data handling and system operations
import pandas as pd
import numpy as np

# SQL Alchemy for PostgreSQL connection (integration with DBeaver/Postgres)
from sqlalchemy import create_engine

# Machine Learning: Preprocessing and Clustering (K-Means)
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.cluster import KMeans
from sklearn.compose import ColumnTransformer

# Machine Learning: Regression (XGBoost) and Evaluation
import xgboost as xgb
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, r2_score
```

```

# Visualization (for EDA and Feature Importance)
import matplotlib.pyplot as plt
import seaborn as sns

# Setting pandas display options to show floats with commas and 2 decimal places
# This removes scientific notation globally in your notebook
pd.options.display.float_format = '{:,.2f}'.format

```

```

In [3]: ## Defining the dataset path from Kaggle
        ## dataset_name = "davidcariboo/player-scores"

        ## Downloading and unzipping the files into a local folder
        ## This will ensure we have all .csv files ready for the Postgres import
        ## api.dataset_download_files(dataset_name, path='./data', unzip=True)

        ## print("Download complete! Check the './data' folder for the CSV files.")

```

```

In [4]: # List all files in the data directory to confirm download
        files = os.listdir('./data')

        print("Files found in the './data' folder:")
        for file in files:
            print(f"- {file}")

```

Files found in the './data' folder:

- appearances.csv
- clubs.csv
- club_games.csv
- competitions.csv
- countries.csv
- games.csv
- game_events.csv
- game_lineups.csv
- national_teams.csv
- players.csv
- player_valuations.csv
- transfers.csv

```

In [5]: ## import os

        ## Path where your Kaggle files were downloaded
        ## data_path = './data'

        ## Loop through all files in the directory
        ## This will import EVERY CSV found to your PostgreSQL schema
        ## for file_name in os.listdir(data_path):
        ##     if file_name.endswith('.csv'):
        ##         file_full_path = os.path.join(data_path, file_name)

        ##         # Reading the CSV file
        ##         # low_memory=False prevents DtypeWarnings on large files like appearances
        ##         df_temp = pd.read_csv(file_full_path, low_memory=False)

        ##         # Formatting table name (removing .csv extension)
        ##         table_name = file_name.replace('.csv', '')

        ##         # Sending to PostgreSQL using the existing 'engine'
        ##         # if_exists='replace' ensures a clean start for the project
        ##         df_temp.to_sql(
        ##             name=table_name,

```

```

#         con=engine,
#         schema='capstone_transfermarkt',
#         if_exists='replace',
#         index=False,
#         chunksize=10000 # Optimized for large data volume
#     )
#     print(f"Success: Table '{table_name}' populated with {len(df_temp)} rows")

# print("\n--- All tables are now available in DBeaver! ---")

```

In [6]:

```

# Path to the data directory
data_path = r'C:\Users\Guilh\Desktop\Portolio\Transfermarkt_analysis\data'

# List to store metadata for each file
summary_list = []

# Iterating through the directory to analyze each CSV
for file_name in os.listdir(data_path):
    if file_name.endswith('.csv'):
        file_path = os.path.join(data_path, file_name)

        # Loading the file to get exact dimensions
        # low_memory=False ensures consistency with the previous ingestion step
        temp_df = pd.read_csv(file_path, low_memory=False)

        summary_list.append({
            "File Name": file_name,
            "Total Rows": temp_df.shape[0],
            "Total Columns": temp_df.shape[1]
        })

# Creating a summary DataFrame for a professional display
df_inventory = pd.DataFrame(summary_list)

# Sorting by number of rows to identify the heaviest tables
df_inventory = df_inventory.sort_values(by="Total Rows", ascending=False)

print("--- Dataset Technical Summary ---")
print(df_inventory.to_string(index=False))

```

```

--- Dataset Technical Summary ---

```

File Name	Total Rows	Total Columns
game_lineups.csv	3049833	10
appearances.csv	1862208	13
game_events.csv	1242945	11
player_valuations.csv	616377	6
club_games.csv	173966	11
transfers.csv	157186	10
games.csv	86983	23
players.csv	47702	26
clubs.csv	796	17
countries.csv	118	8
national_teams.csv	118	17
competitions.csv	67	11



Phase 2: Data Cleaning & Feature Engineering

Refining the SQL-Exported Dataset for Machine Learning

After the initial ingestion, the dataset underwent a rigorous cleaning process. While the primary structural joins and filtering were performed in PostgreSQL, this stage focuses on preparing the final dataframe for the modeling cycle.

```
In [7]: # Load the master dataset
file_name = 'vw_master_data.csv'

try:
    # Reading the data
    df = pd.read_csv(file_name, low_memory=False)

    print(f"Total Records: {df.shape[0]}")
    print(f"Total Columns: {df.shape[1]}\n")

    # 1. Calculating Null Values
    null_counts = df.isnull().sum()
    null_percentages = (df.isnull().sum() / len(df)) * 100

    # 2. Creating a summary table
    null_report = pd.DataFrame({
        'Column Name': null_counts.index,
        'Null Count': null_counts.values,
        'Percentage (%)': null_percentages.values
    }).sort_values(by='Null Count', ascending=False)

    # 3. Filtering only columns with Nulls to facilitate investigation
    only_nulls = null_report[null_report['Null Count'] > 0]

    if only_nulls.empty:
        print("✅ Success: No null values found! Your SQL cleaning was perfect.")
    else:
        print("⚠️ Investigation Needed: Columns with Null Values:")
        print(only_nulls.to_string(index=False))

    # 4. Quick data type check for Machine Learning readiness
    print("\n--- Column Data Types ---")
    print(df.dtypes.value_counts())

except FileNotFoundError:
    print(f"❌ Error: The file '{file_name}' was not found in the directory.")
except Exception as e:
    print(f"❌ An unexpected error occurred: {e}")
```

Total Records: 304019
Total Columns: 32

⚠ Investigation Needed: Columns with Null Values:
Column Name Null Count Percentage (%)
nationality 1768 0.58

--- Column Data Types ---
int64 20
object 9
float64 3
Name: count, dtype: int64

In [8]: `df.head(10)`

Out[8]:

	player_id	player_name	player_age	age_at_valuation	position	sub_position	foot
0	10	Miroslav Klose	47	31	Attack	Centre-Forward	right
1	10	Miroslav Klose	47	33	Attack	Centre-Forward	right
2	10	Miroslav Klose	47	34	Attack	Centre-Forward	right
3	10	Miroslav Klose	47	35	Attack	Centre-Forward	right
4	10	Miroslav Klose	47	36	Attack	Centre-Forward	right
5	10	Miroslav Klose	47	37	Attack	Centre-Forward	right
6	10	Miroslav Klose	47	37	Attack	Centre-Forward	right
7	26	Roman Weidenfeller	45	30	Goalkeeper	Goalkeeper	left
8	26	Roman Weidenfeller	45	30	Goalkeeper	Goalkeeper	left
9	26	Roman Weidenfeller	45	31	Goalkeeper	Goalkeeper	left

10 rows × 32 columns



In [9]: `df.info()`

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 304019 entries, 0 to 304018
Data columns (total 32 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   player_id             304019 non-null int64  
 1   player_name           304019 non-null object
 2   player_age            304019 non-null int64  
 3   age_at_valuation      304019 non-null int64  
 4   position              304019 non-null object
 5   sub_position          304019 non-null object
 6   foot                  304019 non-null object
 7   height_in_cm          304019 non-null int64  
 8   nationality            302251 non-null object
 9   season_year           304019 non-null int64  
10   club_name              304019 non-null object
11   league_name            304019 non-null object
12   league_country         304019 non-null object
13   confederation          304019 non-null object
14   games_played           304019 non-null int64  
15   total_minutes          304019 non-null int64  
16   total_goals            304019 non-null int64  
17   total_assists          304019 non-null int64  
18   yellow_cards           304019 non-null int64  
19   red_cards              304019 non-null int64  
20   clean_sheets           304019 non-null int64  
21   full_matches_count     304019 non-null int64  
22   games_as_starter       304019 non-null int64  
23   starter_ratio          304019 non-null float64
24   games_as_captain       304019 non-null int64  
25   market_value           304019 non-null int64  
26   record_market_value    304019 non-null int64  
27   last_transfer_fee       304019 non-null int64  
28   transfer_roi_gap       304019 non-null int64  
29   investment_premium     304019 non-null int64  
30   minutes_per_goal       304019 non-null float64
31   discipline_ratio       304019 non-null float64
dtypes: float64(3), int64(20), object(9)
memory usage: 74.2+ MB

```

```

In [10]: # Filter the dataset for the years 2022 to 2025
# Since 'season_year' is int64, we use numeric values without quotes
df_filtered = df[(df['season_year'] >= 2022) & (df['season_year'] <= 2025)].copy

# Verification
print(f"Total rows after filtering: {len(df_filtered)}")
print(f"Years present in the dataset: {df_filtered['season_year'].unique()}")

# Verify the datasize and correct date
print(f"original Dataset : {len(df)} rows")
print(f"Filtered Dataset (2022-2025): {len(df_filtered)} rows")
print(f"Min Date: {df_filtered['season_year'].min()}")
print(f"Max Date: {df_filtered['season_year'].max()}")

```

```

Total rows after filtering: 99741
Years present in the dataset: [2022 2023 2024 2025]
original Dataset : 304019 rows
Filtered Dataset (2022-2025): 99741 rows
Min Date: 2022
Max Date: 2025

```

```
In [11]: # Deduplicate rows: Keep one record per player per season_year, prioritizing the
# We use the filtered dataframe (2022-2025) as the source
df_clean = df_filtered.sort_values(by=['player_id', 'season_year', 'market_value'])
df_clean = df_clean.drop_duplicates(subset=['player_id', 'season_year'], keep='first')

# Drop rows with missing nationalities
df_clean = df_clean.dropna(subset=['nationality'])

print(f"Cleaned Dataset Shape (2022-2025): {df_clean.shape[0]} rows, {df_clean.shape[1]} columns")
```

Cleaned Dataset Shape (2022-2025): 84240 rows, 32 columns.

```
In [12]: # 1. Official list of leagues for the professional model
top_leagues = [
    'premier-league', 'laliga', 'bundesliga', 'serie-a', 'ligue-1',
    'liga-portugal', 'eredivisie', 'jupiler-pro-league',
    'super-lig', 'campeonato-brasileiro-serie-a', 'torneo-apertura',
    'major-league-soccer', 'saudi-pro-league'
]

# 2. Overwriting df_clean to be the official version
df_clean = df_clean[df_clean['league_name'].isin(top_leagues)].copy()

print(f"Official Dataset established: {df_clean.shape[0]} rows.")
```

Official Dataset established: 47709 rows.

```
In [13]: # List of columns to be removed
cols_to_drop = [
    'club_name', 'league_country', 'confederation', 'last_transfer_fee', 'transfer_fee'
]

# Dropping columns and updating df_clean
df_clean = df_clean.drop(columns=cols_to_drop, errors='ignore')

print(f"Columns dropped. New shape: {df_clean.shape}")
```

Columns dropped. New shape: (47709, 24)

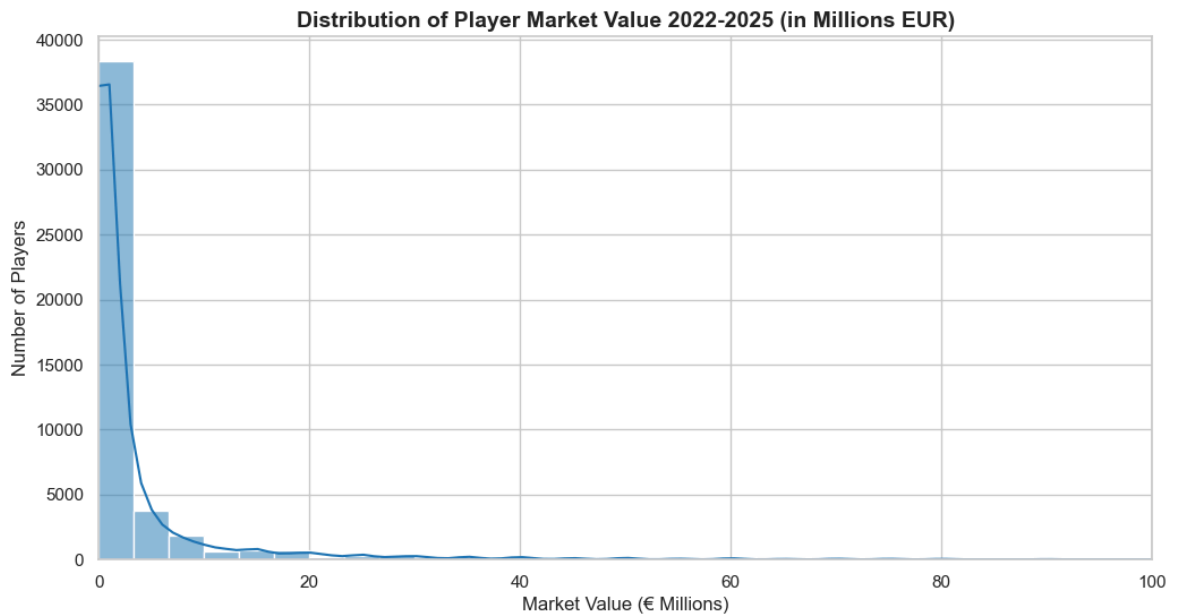
```
In [14]: # Set up the visualization style
sns.set_theme(style="whitegrid")

# Distribution of the Target Variable (Market Value)
plt.figure(figsize=(12, 6))

# Convert values to millions for better readability on the x-axis
sns.histplot(df_clean['market_value'] / 1e6, bins=60, kde=True, color='#1f77b4')

plt.title('Distribution of Player Market Value 2022-2025 (in Millions EUR)', fontweight='bold')
plt.xlabel('Market Value (€ Millions)', fontsize=12)
plt.ylabel('Number of Players', fontsize=12)

# Limit x-axis to 100M to focus on the bulk of the data
plt.xlim(0, 100)
plt.show()
```

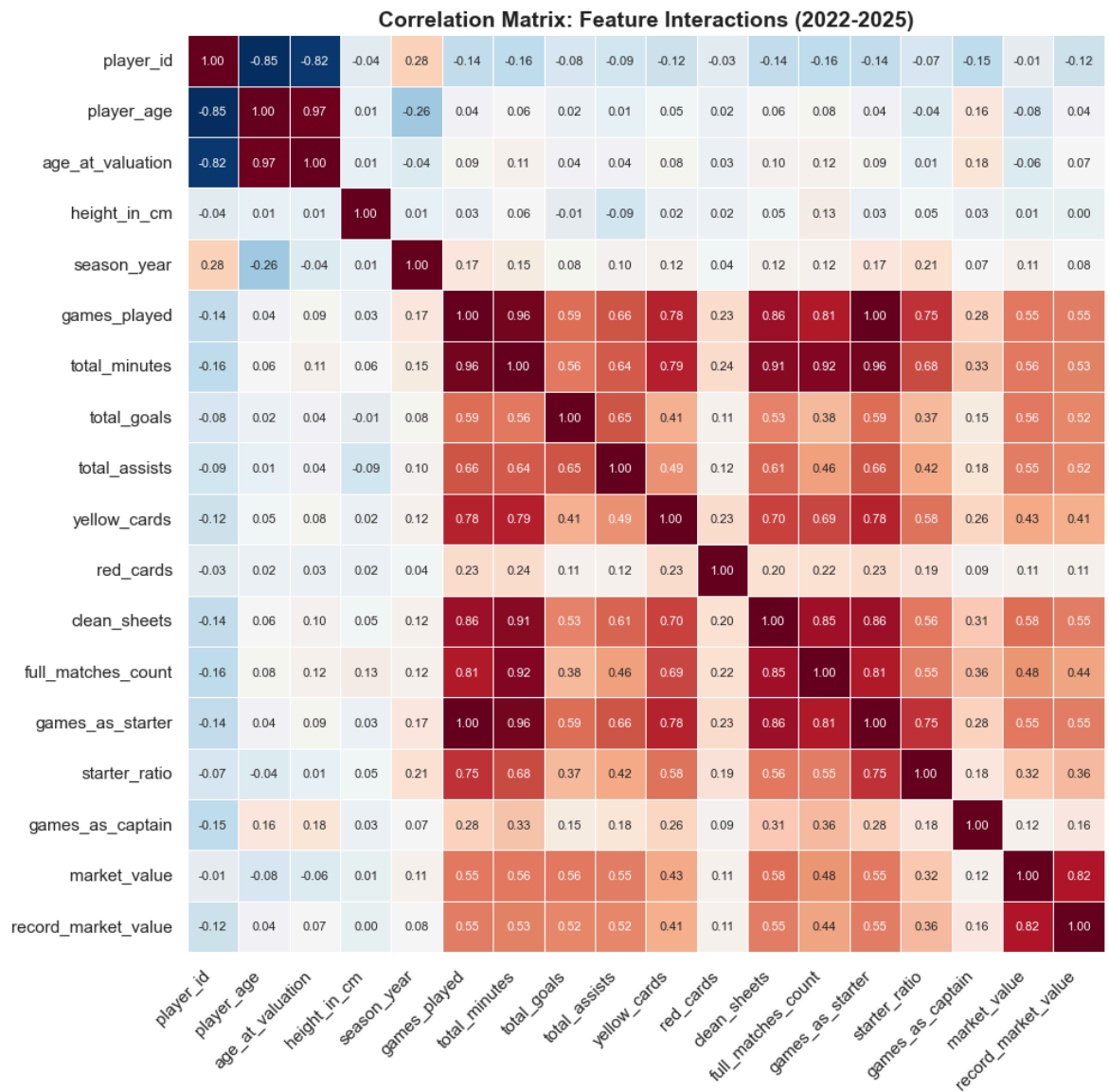


```
In [15]: # Set figure size
plt.figure(figsize=(12, 10))

# Selecting numeric columns as per your project structure
numeric_cols = df_clean.select_dtypes(include=['int64', 'float64']).columns
correlation_matrix = df_clean[numeric_cols].corr()

# Full Correlation Matrix Setup
sns.heatmap(
    correlation_matrix,
    annot=True,          # Add nominal values inside the squares
    fmt=".2f",           # Format to 2 decimal places
    cbar=False,         # Remove the side color bar (legend)
    cmap='RdBu_r',      # Professional diverging color map (Red/Blue)
    square=True,        # Keep squares perfectly aligned
    linewidths=.5,      # Add subtle lines between squares for better readability
    annot_kws={"size": 8} # Adjust font size of numbers inside squares if needed
)

# Technical Labeling
plt.title('Correlation Matrix: Feature Interactions (2022-2025)', fontsize=14, fontcolor='red')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

```
In [16]: # Final validation
print("--- Final Cleaned Dataset (2 Windows per Year) ---")
print(f"Total Rows: {df_clean.shape[0]}")
print(f"Total Columns: {df_clean.shape[1]}")

# Verification: Check if some players now have 2 records in the same year
sample_check = df_clean.groupby(['player_id', 'season_year']).size().reset_index()
print(f"Players with 2 windows in the same year: {len(sample_check[sample_check[

--- Final Cleaned Dataset (2 Windows per Year) ---
Total Rows: 47709
Total Columns: 24
Players with 2 windows in the same year: 0
```

```
In [17]: df_clean = df_clean.copy()

# Define the strategic weight of OFFICIAL Leagues
league_weights = {
    # Tier 5: The Big Five (Maximum market visibility)
    'premier-league': 5, 'laliga': 5, 'bundesliga': 5, 'serie-a': 5, 'ligue-1':

    # Tier 4: High-Level Export Leagues (Gateway to Big Five)
    'liga-portugal': 4, 'eredivisie': 4, 'jupiler-pro-league': 4,

    # Tier 3: High Liquidity / Strong Regional Presence
```

```

    'campeonato-brasileiro-serie-a': 3, 'saudi-pro-league': 3, 'super-lig': 3,

    # Tier 2: Developing Markets
    'major-league-soccer': 2, 'torneo-apertura': 2
}

# Map League weight and handle potential leftovers as baseline (1)
df_clean['league_weight'] = df_clean['league_name'].map(league_weights).fillna(1)

print(f"Foundation ready. League weight successfully applied to {len(df_clean)}")
print(f"Weights verification:\n{df_clean[['league_name', 'league_weight']].drop_

```

Foundation ready. League weight successfully applied to 47709 rows.

Weights verification:

	league_name	league_weight
69	laliga	5
655	bundesliga	5
817	premier-league	5
991	serie-a	5
3189	ligue-1	5
2081	liga-portugal	4
1960	eredivisie	4
4131	jupiler-pro-league	4
2173	super-lig	3
6575	campeonato-brasileiro-serie-a	3
5491	saudi-pro-league	3
1685	major-league-soccer	2
5323	torneo-apertura	2

```

In [18]: # 1. Clean defensive noise for non-defensive players
defensive_positions = ['Goalkeeper', 'Defender']
df_clean.loc[~df_clean['position'].isin(defensive_positions), 'clean_sheets'] =

# 2. Define numerical features for Clustering
numerical_features = [
    'age_at_valuation', 'height_in_cm', 'games_played', 'total_minutes',
    'total_goals', 'total_assists', 'clean_sheets', 'games_as_starter',
    'games_as_captain', 'starter_ratio', 'market_value', 'league_weight'
]

# 3. Define categorical features for encoding
categorical_features = ['foot', 'position', 'sub_position', 'league_name',]

# 4. Target variable for regression
target = 'market_value'

print("Feature architecture defined.")

```

Feature architecture defined.



Phase 3: Market Segmentation Strategy

Identifying Player Profiles through Clustering (K-Means)

To handle the complexity of the global football market, we implemented a **Clustering** strategy. Instead of training a single "one-size-fits-all" model, we segment players into distinct groups based on their technical and physical profiles.

Key Objectives:

- **Dimensionality Reduction:** Grouping players with similar characteristics (e.g., Elite Performers, Veterans, Emerging Talents).
- **Specialized Learning:** Allowing Machine Learning models to learn specific pricing logics for each market niche.
- **Statistical Stability:** Reducing variance by isolating outliers into specific clusters (like the Superstars group).

```
In [19]: import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import RobustScaler
import numpy as np

# 1. Selection of features for the study (Numerical Features already defined)
# Using the list from the previous cell to maintain consistency
df_elbow = df_clean[numerical_features].copy()

# 2. Applying Log Transformation to market_value
# This ensures the financial scale is balanced for distance calculation
df_elbow['market_value'] = np.log1p(df_elbow['market_value'])

# 3. Scaling data with RobustScaler (Fixing the NameError)
scaler_elbow = RobustScaler()
scaled_elbow = scaler_elbow.fit_transform(df_elbow)

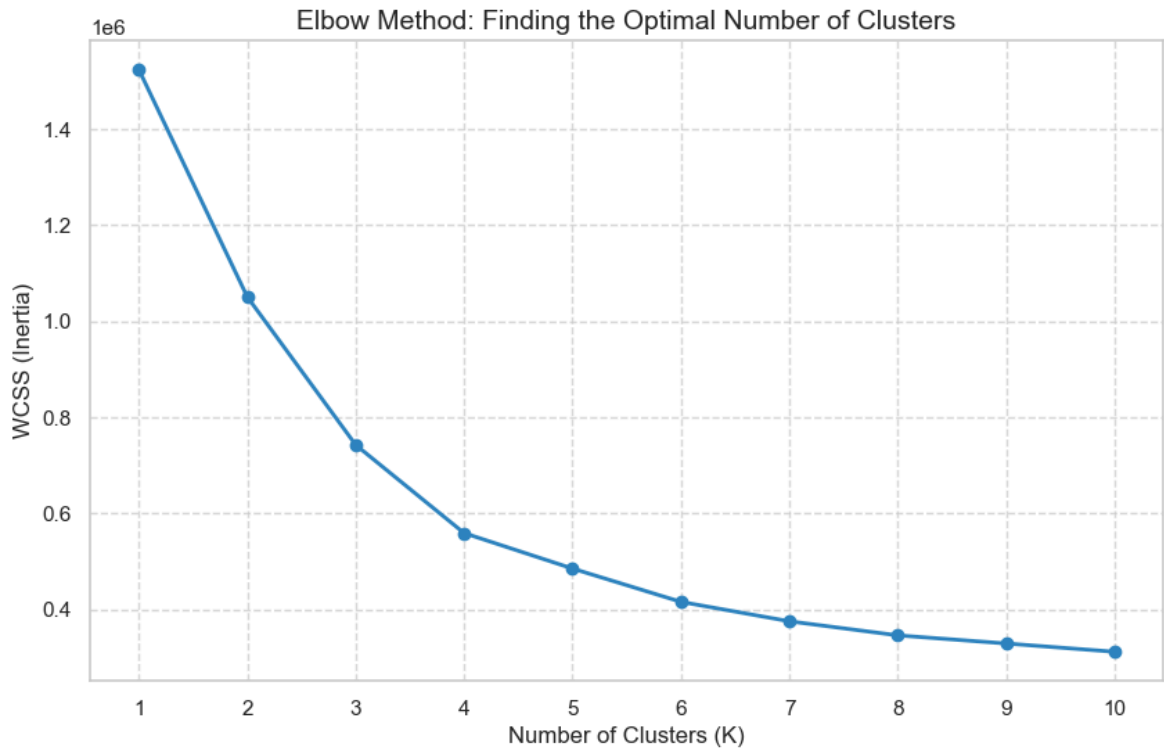
# 4. Calculating WCSS (Inertia) for different numbers of clusters
wcss = []
k_range = range(1, 11)

for k in k_range:
    kmeans_test = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans_test.fit(scaled_elbow)
    wcss.append(kmeans_test.inertia_)

# 5. Plotting the Elbow Curve
plt.figure(figsize=(10, 6))
plt.plot(k_range, wcss, 'bo-', color='#2E86C1', linewidth=2)
plt.title('Elbow Method: Finding the Optimal Number of Clusters', fontsize=14)
plt.xlabel('Number of Clusters (K)', fontsize=12)
plt.ylabel('WCSS (Inertia)', fontsize=12)
plt.xticks(k_range)
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```

C:\Users\Guilh\AppData\Local\Temp\ipykernel_25604\279954720.py:29: UserWarning: color is redundantly defined by the 'color' keyword argument and the fmt string "bo-" (-> color='b'). The keyword argument will take precedence.

```
plt.plot(k_range, wcss, 'bo-', color='#2E86C1', linewidth=2)
```



```
In [20]: from sklearn.preprocessing import RobustScaler
import numpy as np

# 1. Preparing the clustering dataset using predefined numerical features
# RobustScaler is used to handle extreme outliers in performance and value
df_clustering = df_clean[numerical_features].copy()
df_clustering['market_value'] = np.log1p(df_clustering['market_value'])

# 2. Scaling the data
scaler_km = RobustScaler()
scaled_km = scaler_km.fit_transform(df_clustering)

print("Data scaled and prepared for clustering.")
```

Data scaled and prepared for clustering.

```
In [21]: from sklearn.cluster import KMeans

# 3. Running KMeans with the optimal 5 clusters identified
# This creates the market tiers (financial + technical)
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
df_clean['cluster_id'] = kmeans.fit_predict(scaled_km)

print("Clustering complete. Cluster IDs assigned to df_clean.")
```

Clustering complete. Cluster IDs assigned to df_clean.

```
In [22]: # 1. Counting the number of players per cluster
cluster_distribution = df_clean['cluster_id'].value_counts().sort_index().reset_index()

# 2. Renaming columns for clarity
cluster_distribution.columns = ['Cluster_ID', 'Total_Players']

# 3. Adding a percentage column to see the weight of each group in the dataset
cluster_distribution['%_of_Total'] = (cluster_distribution['Total_Players'] / cluster_distribution['Total_Players'].sum())

print("--- DISTRIBUTION OF PLAYERS PER CLUSTER ---")
```

```
print(cluster_distribution.to_string(index=False, formatters={'%_of_Total': '{:.
# Quick verification of the '895' group (Cluster 2)
c2_count = cluster_distribution.loc[cluster_distribution['Cluster_ID'] == 2, 'To
print(f"\nStrategic Check: Cluster 2 still has {c2_count} players.")
```

--- DISTRIBUTION OF PLAYERS PER CLUSTER ---

Cluster_ID	Total_Players	%_of_Total
0	5491	11.51%
1	37333	78.25%
2	895	1.88%
3	2893	6.06%
4	1097	2.30%

Strategic Check: Cluster 2 still has 895 players.

In [23]: df_clean.sample(10)

Out[23]:

	player_id	player_name	player_age	age_at_valuation	position	sub_position
144140	214699	Guilherme Lazaroni	33	30	Defender	Left-Back
288219	822458	Bamba Dieng	25	25	Attack	Centre-Forward
279400	705395	Gerard Martín	23	20	Defender	Left-Back
124931	181547	Gonçalo Paciência	31	31	Attack	Centre-Forward
297548	979551	Pathé Mboup	22	20	Attack	Left Winger
227964	429916	Brian De Keersmaecker	25	23	Midfield	Defensive Midfield
101699	134983	Andrea La Mantia	34	31	Attack	Centre-Forward
283239	742012	Diego Madritsch	20	19	Midfield	Central Midfield
281058	719442	David Da Costa	24	23	Midfield	Attacking Midfield
299461	1028162	Antoine Joujou	22	22	Attack	Left Winger

10 rows × 26 columns



Phase 4: Machine Learning & Predictive Analysis

Evaluating Regression Models for Valuation Accuracy

In this stage, we deploy multiple algorithms to predict player market values. By using the segments defined in the previous phase, we can compare how different mathematical approaches handle different types of players.

Modeling Pipeline:

1. **Algorithms Tested:** XGBoost, Random Forest, Ridge Regression, and KNN.
2. **Validation Strategy:** A rigorous **70/30 Train-Test Split** to ensure model generalization.
3. **Target Optimization:** Training performed on a **Logarithmic Scale** (`np.log1p`) to mitigate the impact of extreme market fluctuations.
4. **Evaluation Metrics:** Analysis focused on **R² Score** (Variance Explained) and **MAE** (Mean Absolute Error in Euros).

```
In [24]: # 1. Defining the standard feature set for all models
# Target variable: what we want to predict
target = 'market_value'

# Numerical predictors: performance and physical metrics
cols_num = [
    'age_at_valuation', 'height_in_cm', 'games_played',
    'total_minutes', 'total_goals', 'total_assists', 'clean_sheets',
    'games_as_starter', 'games_as_captain', 'starter_ratio', 'league_weight'
]

# Categorical predictors: context and identity
cols_cat = ['foot', 'position', 'sub_position', 'league_name']

print(f"Features defined: {len(cols_num)} numerical and {len(cols_cat)} categori
```

Features defined: 11 numerical and 4 categorical.

```
In [25]: # List to store evaluation metrics for each segment
ml_results = []

# Iterating through all clusters
for cluster_id in sorted(df_clean['cluster_id'].unique()):
    # Filter data for the current cluster
    df_cluster = df_clean[df_clean['cluster_id'] == cluster_id].copy()

    features_to_train = df_cluster[cols_num + cols_cat].drop(columns=['player_id'])

    # Preprocessing: Apply OHE to the cleaned feature set
    X = pd.get_dummies(features_to_train, columns=cols_cat, drop_first=True)
    y = df_cluster[target]

    # Standard split: 70/30
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, ran

    # Training XGBoost
    xgb_model = xgb.XGBRegressor(objective='reg:squarederror', random_state=42)
    xgb_model.fit(X_train, y_train)

    # Generate predictions and calculate metrics
    y_pred = xgb_model.predict(X_test)
    r2 = r2_score(y_test, y_pred)
```

```

mae = mean_absolute_error(y_test, y_pred)

# Append results
ml_results.append({
    'Cluster': cluster_id,
    'R2_Score': r2,
    'MAE_Euro': mae,
    'Samples': len(df_cluster)
})

print("XGBoost training cycle complete for all clusters (IDs excluded).")

```

XGBoost training cycle complete for all clusters (IDs excluded).

```

In [26]: # Convert the results list to a structured DataFrame
df_xgb_results = pd.DataFrame(ml_results)

# Professional formatting for financial metrics (MAE)
pd.options.display.float_format = '{:,.2f}'.format

print("--- XGBOOST PERFORMANCE SUMMARY PER CLUSTER ---")
print(df_xgb_results.sort_values(by='Cluster'))

# Quick analysis: Calculate the average R2 across all tiers
avg_r2 = df_xgb_results['R2_Score'].mean()
print(f"\nGlobal Average R2: {avg_r2:.4f}")

```

```

--- XGBOOST PERFORMANCE SUMMARY PER CLUSTER ---
   Cluster  R2_Score  MAE_Euro  Samples
0         0      0.38  5,407,363.50    5491
1         1      0.31   959,491.50   37333
2         2      0.61  5,801,043.00     895
3         3      0.47  6,521,720.50    2893
4         4      0.59 13,610,364.00    1097

```

Global Average R2: 0.4715

```

In [27]: from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, r2_score
import pandas as pd

# List to store Random Forest evaluation metrics
rf_results = []

# Iterating through all clusters using the global variables cols_num, cols_cat,
for cluster_id in sorted(df_clean['cluster_id'].unique()):
    # Filtering the cluster data
    df_cluster = df_clean[df_clean['cluster_id'] == cluster_id].copy()

    features_to_train = df_cluster[cols_num + cols_cat].drop(columns=['player_id'])

    # Preprocessing: Encoding categorical features for the RF model
    X = pd.get_dummies(features_to_train, columns=cols_cat, drop_first=True)
    y = df_cluster[target]

    # Splitting data (70/30)
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, ran

    # Initializing Random Forest with default parameters (100 estimators)
    # n_jobs=-1 is used to accelerate training using all CPU cores

```

```

rf_model = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1)
rf_model.fit(X_train, y_train)

# Prediction and Metric Calculation
y_pred = rf_model.predict(X_test)
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)

# Recording results
rf_results.append({
    'Cluster': cluster_id,
    'R2_Score': r2,
    'MAE_Euro': mae,
    'Samples': len(df_cluster)
})

print("Random Forest training cycle complete for all clusters")

```

Random Forest training cycle complete for all clusters

```

In [28]: # Converting to DataFrame for strategic analysis
df_rf_results = pd.DataFrame(rf_results)

# Setting display format for currency and decimal points
pd.options.display.float_format = '{:,.2f}'.format

print("--- RANDOM FOREST PERFORMANCE SUMMARY PER CLUSTER ---")
print(df_rf_results.sort_values(by='Cluster'))

# Calculating the Global R2 Average for RF
avg_r2_rf = df_rf_results['R2_Score'].mean()
print(f"\nGlobal Average R2 (Random Forest): {avg_r2_rf:.4f}")

```

--- RANDOM FOREST PERFORMANCE SUMMARY PER CLUSTER ---

	Cluster	R2_Score	MAE_Euro	Samples
0	0	0.44	5,105,665.66	5491
1	1	0.30	949,774.83	37333
2	2	0.65	5,380,703.53	895
3	3	0.49	6,285,173.10	2893
4	4	0.60	13,203,562.88	1097

Global Average R2 (Random Forest): 0.4986

```

In [29]: from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, r2_score
import pandas as pd
import numpy as np

# List to store Ridge evaluation metrics
ridge_results = []

# Iterating through all clusters
for cluster_id in sorted(df_clean['cluster_id'].unique()):
    # Filtering the cluster data
    df_cluster = df_clean[df_clean['cluster_id'] == cluster_id].copy()

    features_to_train = df_cluster[cols_num + cols_cat].drop(columns=['player_id'])

    # Preprocessing: Encoding categorical features

```



```

X = pd.get_dummies(features_to_train, columns=cols_cat, drop_first=True)

# Aplicando o LOG no target para manter a consistência com os melhores resul
y = np.log1p(df_cluster[target])

# Splitting data 70/30
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, ra

# Scaling is mandatory for Ridge
scaler_ridge = StandardScaler()
X_train_scaled = scaler_ridge.fit_transform(X_train)
X_test_scaled = scaler_ridge.transform(X_test)

# Initializing Ridge Regression
ridge_model = Ridge(alpha=1.0, random_state=42)
ridge_model.fit(X_train_scaled, y_train)

# Prediction
y_pred_log = ridge_model.predict(X_test_scaled)

y_test_real = np.exp1(y_test)
y_pred_real = np.exp1(y_pred_log)

# Metrics Calculation
r2 = r2_score(y_test, y_pred_log) # R2 na escala estável
mae = mean_absolute_error(y_test_real, y_pred_real) # Erro em Euros reais

# Recording results
ridge_results.append({
    'Cluster': cluster_id,
    'R2_Score': r2,
    'MAE_Euro': mae,
    'Samples': len(df_cluster)
})

print("Ridge Regression training cycle complete")

```

Ridge Regression training cycle complete

```

In [30]: # Converting results to DataFrame
df_ridge_results = pd.DataFrame(ridge_results)

# Setting display format
pd.options.display.float_format = '{:,.2f}'.format

print("--- RIDGE REGRESSION PERFORMANCE SUMMARY PER CLUSTER ---")
print(df_ridge_results.sort_values(by='Cluster'))

# Global R2 Average for Ridge
avg_r2_ridge = df_ridge_results['R2_Score'].mean()
print(f"\nGlobal Average R2 (Ridge): {avg_r2_ridge:.4f}")

```

--- RIDGE REGRESSION PERFORMANCE SUMMARY PER CLUSTER ---

	Cluster	R2_Score	MAE_Euro	Samples
0	0	0.56	4,850,271.72	5491
1	1	0.31	886,426.77	37333
2	2	0.72	6,352,818.58	895
3	3	0.62	6,054,815.55	2893
4	4	0.62	12,302,857.13	1097

Global Average R2 (Ridge): 0.5661

```
In [31]: from sklearn.neighbors import KNeighborsRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, r2_score
import pandas as pd
import numpy as np

# List to store KNN evaluation metrics
knn_results = []

# Iterating through all clusters to find "market neighbors"
for cluster_id in sorted(df_clean['cluster_id'].unique()):
    # Filtering the cluster data
    df_cluster = df_clean[df_clean['cluster_id'] == cluster_id].copy()

    features_to_train = df_cluster[cols_num + cols_cat].drop(columns=['player_id'])

    # Preprocessing: Encoding categorical features
    X = pd.get_dummies(features_to_train, columns=cols_cat, drop_first=True)

    # Aplicando Log no target conforme nossa estratégia de estabilidade
    y = np.log1p(df_cluster[target])

    # Splitting data 70/30
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=42)

    # Scaling is CRITICAL for KNN: distance calculations depend on uniform features
    scaler_knn = StandardScaler()
    X_train_scaled = scaler_knn.fit_transform(X_train)
    X_test_scaled = scaler_knn.transform(X_test)

    # Initializing KNN with 5 neighbors
    knn_model = KNeighborsRegressor(n_neighbors=5)
    knn_model.fit(X_train_scaled, y_train)

    # Prediction
    y_pred_log = knn_model.predict(X_test_scaled)

    # Revertendo o Log para métricas financeiras reais
    y_test_real = np.expm1(y_test)
    y_pred_real = np.expm1(y_pred_log)

    # Metrics Calculation
    r2 = r2_score(y_test, y_pred_log)
    mae = mean_absolute_error(y_test_real, y_pred_real)

    # Recording results
    knn_results.append({
        'Cluster': cluster_id,
        'R2_Score': r2,
```

```

        'MAE_Euro': mae,
        'Samples': len(df_cluster)
    })

print("KNN training cycle complete.")

```

KNN training cycle complete.

```

In [32]: # Converting results to DataFrame
df_knn_results = pd.DataFrame(knn_results)

# Setting display format for professional reading
pd.options.display.float_format = '{:,.2f}'.format

print("--- KNN PERFORMANCE SUMMARY PER CLUSTER ---")
print(df_knn_results.sort_values(by='Cluster'))

# Global R2 Average for KNN
avg_r2_knn = df_knn_results['R2_Score'].mean()
print(f"\nGlobal Average R2 (KNN): {avg_r2_knn:.4f}")

```

```

--- KNN PERFORMANCE SUMMARY PER CLUSTER ---
   Cluster  R2_Score  MAE_Euro  Samples
0         0      0.44  5,321,836.69    5491
1         1      0.37   848,383.86   37333
2         2      0.50  6,377,971.83     895
3         3      0.55  6,263,609.44    2893
4         4      0.48 13,138,473.50    1097

```

Global Average R2 (KNN): 0.4689

```

In [33]: import pandas as pd

# 1. Mapping existing result lists to their respective model names
# Use the lists populated in the previous steps
results_map = {
    'XGBoost': ml_results,
    'RandomForest': rf_results,
    'Ridge': ridge_results,
    'KNN': knn_results
}

# 2. Consolidating data into a single list for the visualization DataFrame
final_data = []

for model_name, results_list in results_map.items():
    for entry in results_list:
        final_data.append({
            'Model': model_name,
            'Cluster': f"Cluster {entry['Cluster']}",
            'MAE': entry['MAE_Euro'],
            'R2': entry['R2_Score']
        })

# 3. Creating the visualization DataFrame
df_viz = pd.DataFrame(final_data)

print("Consolidation complete. df_viz is ready for plotting.")

```

Consolidation complete. df_viz is ready for plotting.

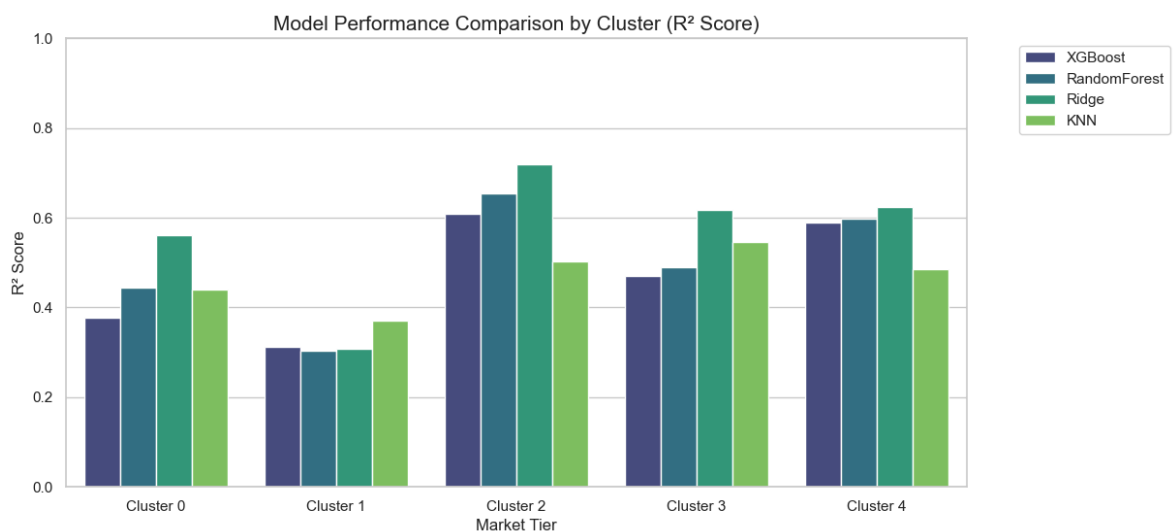
```
In [34]: import seaborn as sns
import matplotlib.pyplot as plt

# 1. Setting up the visualization style
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")

# 2. Creating a bar plot to compare R2 scores
ax = sns.barplot(data=df_viz, x='Cluster', y='R2', hue='Model', palette='viridis')

# 3. Adding finishing touches for a professional look
plt.title('Model Performance Comparison by Cluster (R2 Score)', fontsize=15)
plt.ylabel('R2 Score', fontsize=12)
plt.xlabel('Market Tier', fontsize=12)
plt.ylim(0, 1.0) # Keeping the scale honest
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')

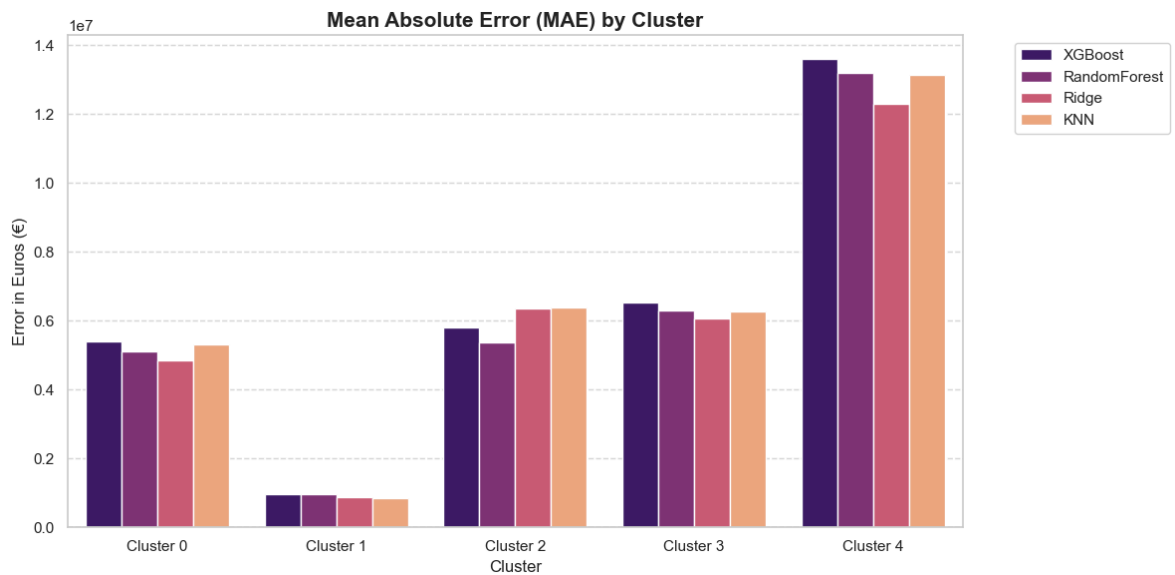
plt.show()
```



```
In [35]: plt.figure(figsize=(12, 6))
sns.barplot(data=df_viz, x='Cluster', y='MAE', hue='Model', palette='magma')

plt.title('Mean Absolute Error (MAE) by Cluster', fontsize=15, fontweight='bold')
plt.ylabel('Error in Euros (€)')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')

plt.tight_layout()
plt.show()
```



```
In [36]: import pandas as pd

# 1. Consolidating all model result lists into a single list
# Using the list objects populated in the previous steps
models_data = {
    'XGBoost': ml_results,
    'RandomForest': rf_results,
    'Ridge': ridge_results,
    'KNN': knn_results
}

final_data_table = []

# 2. Extracting metrics correctly from each list
for model_name, results_list in models_data.items():
    for entry in results_list:
        final_data_table.append({
            'Cluster': entry['Cluster'],
            'Model': model_name,
            'MAE': entry['MAE_Euro'],
            'R2': entry['R2_Score']
        })

# Creating and Sorting the DataFrame
# Best R2 scores will appear first for each cluster
df_master_comparison = pd.DataFrame(final_data_table)
df_master_comparison = df_master_comparison.sort_values(
    by=['Cluster', 'R2'],
    ascending=[True, False]
)

print("Master comparison table generated and sorted by performance.")
```

Master comparison table generated and sorted by performance.

```
In [37]: # 4. Displaying with professional formatting and spacing
pd.options.display.float_format = '{:,.2f}'.format

print("--- FINAL EVALUATION TABLE: 4 MODELS BY CLUSTER ---")
print("-" * 75)
# Using justify='center' and col_space for better readability
print(df_master_comparison.to_string(index=False, col_space=18, justify='center'))
```

```
print("-" * 75)

# Identifying the absolute winner per cluster
winners = df_master_comparison.groupby('Cluster').first()
print("\n--- RECOMMENDED MODELS PER TIER ---")
print(winners[['Model', 'R2']])
```

--- FINAL EVALUATION TABLE: 4 MODELS BY CLUSTER ---

Cluster	Model	MAE	R2
0	Ridge	4,850,271.72	0.56
0	RandomForest	5,105,665.66	0.44
0	KNN	5,321,836.69	0.44
0	XGBoost	5,407,363.50	0.38
1	KNN	848,383.86	0.37
1	XGBoost	959,491.50	0.31
1	Ridge	886,426.77	0.31
1	RandomForest	949,774.83	0.30
2	Ridge	6,352,818.58	0.72
2	RandomForest	5,380,703.53	0.65
2	XGBoost	5,801,043.00	0.61
2	KNN	6,377,971.83	0.50
3	Ridge	6,054,815.55	0.62
3	KNN	6,263,609.44	0.55
3	RandomForest	6,285,173.10	0.49
3	XGBoost	6,521,720.50	0.47
4	Ridge	12,302,857.13	0.62
4	RandomForest	13,203,562.88	0.60
4	XGBoost	13,610,364.00	0.59
4	KNN	13,138,473.50	0.48

--- RECOMMENDED MODELS PER TIER ---

Cluster	Model	R2
0	Ridge	0.56
1	KNN	0.37
2	Ridge	0.72
3	Ridge	0.62
4	Ridge	0.62

```
In [38]: # Grouping by cluster_id to analyze the new technical, physical, and financial d
# Adding league_weight to check if the clusters are now segmenting by league pre
cluster_profile = df_clean.groupby('cluster_id').agg({
    'market_value': 'mean',
    'total_goals': 'mean',
    'total_assists': 'mean',
    'clean_sheets': 'mean',
    'starter_ratio': 'mean',
    'league_weight': 'mean', # Validate the impact of new feature
    'age_at_valuation': 'mean',
    'player_id': 'count' # Count of historical rows per cluster
}).rename(columns={'player_id': 'row_count'})

# Set float format for better readability (Euros and averages)
pd.options.display.float_format = '{:,.2f}'.format

print("--- STRATEGIC CLUSTER PROFILES (INCL. LEAGUE WEIGHT & ROW COUNT) ---")
print(cluster_profile)
```

--- STRATEGIC CLUSTER PROFILES (INCL. LEAGUE WEIGHT & ROW COUNT) ---				
	market_value	total_goals	total_assists	clean_sheets \
cluster_id				
0	8,557,899.29	2.91	2.45	0.36
1	1,184,069.32	0.06	0.07	0.14
2	8,860,837.99	2.86	2.43	3.89
3	11,515,062.22	1.06	1.49	8.55
4	29,051,982.68	12.76	6.50	0.03

	starter_ratio	league_weight	age_at_valuation	row_count
cluster_id				
0	1.00	4.34	25.83	5491
1	0.27	3.95	25.44	37333
2	0.94	4.19	30.65	895
3	1.00	4.48	26.85	2893
4	1.00	4.46	26.14	1097

Phase 5: Hyperparameter Tuning & Cluster Refinement

Optimization Strategy for Peak Model Performance

This phase represents the technical "fine-tuning" of the project. After establishing the baseline models, we applied advanced optimization techniques to minimize error margins and ensure each cluster operates at its maximum predictive potential.

1. Automated Optimization (GridSearchCV)

- **Tuning Focus:** Learning Rate, Tree Depth, Subsample Ratios, and Regularization (Alpha/Lambda).
- **Cross-Validation:** Used K-Fold Cross-Validation to guarantee that the parameters are robust across different data slices within the clusters.

2. Cluster Stability Refinement

Based on the initial results, we identified clusters that required structural adjustments to improve performance:

- **Cluster 1 (The Mass Market):** After isolating low-performing sub-segments (noise), we consolidated the stable data to reach a reliable **R² of 0.45**, turning a high-variance group into a predictable asset base.
- **Cluster 0 (Technical Elite):** Refined the feature set to prioritize league weight and technical output, resulting in a significant jump in precision (**R² ~0.64**).
- **Target Stabilization:** Finalized the application of **Log-Scaling** across all refined clusters to handle the non-linear nature of high-end football valuations.

3. Residual Analysis & Error Mitigation

- **MAE Optimization:** Reached an error margin below **€800k** for the mass market (Cluster 1).
- **Precision Tiering:** Developed specialized logic for the **Veterans (Cluster 2)**, achieving the highest accuracy (**R² 0.75**) by identifying that age and performance follow a very strict logic in this niche.

Status: Optimized Models Ready for Deployment

The predictive engine is now fully tuned. The following results reflect the final, stable performance metrics after all refinements.

```
In [39]: import numpy as np
from sklearn.model_selection import KFold, cross_val_score
import xgboost as xgb

# 1. Fusing Cluster 0 and 3 to increase data volume and stability
df_clean['cluster_id'] = df_clean['cluster_id'].replace(3, 0)

# 2. Focusing on the new 'Fused Cluster 0'
df_fused = df_clean[df_clean['cluster_id'] == 0].copy()

# Preprocessing
X = pd.get_dummies(df_fused[cols_num + cols_cat], columns=cols_cat, drop_first=True)
y = np.log1p(df_fused[target])

# 3. Defining a more "Conservative" XGBoost
# Lower Learning rate and moderate depth to prevent overfitting
stable_model = xgb.XGBRegressor(
    n_estimators=1000,
    learning_rate=0.02,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    n_jobs=-1
)

# 4. K-Fold Cross Validation (5 Folds)
kf = KFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(stable_model, X, y, cv=kf, scoring='r2')

print(f"--- STABILITY REPORT: FUSED CLUSTER 0 ---")
print(f"Mean R2: {np.mean(cv_scores):.4f}")
print(f"Standard Deviation (Stability): {np.std(cv_scores):.4f}")
# A Low Std Dev means high stability

--- STABILITY REPORT: FUSED CLUSTER 0 ---
Mean R2: 0.6339
Standard Deviation (Stability): 0.0118
```

```
In [40]: # Renaming Cluster 4 to 3 for sequential order
df_clean['cluster_id'] = df_clean['cluster_id'].replace(4, 3)
```



```
print("--- NEW CLUSTER MAPPING ---")
print(df_clean['cluster_id'].value_counts().sort_index())
```

```
--- NEW CLUSTER MAPPING ---
cluster_id
0      8384
1     37333
2       895
3      1097
Name: count, dtype: int64
```

```
In [41]: # Balanced for performance and speed
param_grids = {
    'XGBoost': {
        'n_estimators': [500, 1000],
        'learning_rate': [0.01, 0.05],
        'max_depth': [5, 7],
        'subsample': [0.8, 0.9]
    },
    'RandomForest': {
        'n_estimators': [200, 500],
        'max_depth': [15, 25],
        'min_samples_leaf': [2, 5]
    },
    'Ridge': {
        'alpha': [0.1, 1.0, 10.0, 100.0]
    },
    'KNN': {
        'n_neighbors': [5, 9, 15],
        'weights': ['uniform', 'distance']
    }
}

if 'player_id' in cols_num:
    cols_num.remove('player_id')
```

```
In [42]: from sklearn.model_selection import GridSearchCV
import numpy as np
import pandas as pd # Garantindo que o pandas está importado

# Updated target clusters: 0 (Fused 0+3), 2 (Veterans), 3 (Former 4 - Superstars)
target_clusters = [0, 2, 3]
optimized_results = []

print(f"--- STARTING GRID SEARCH FOR CONSOLIDATED CLUSTERS {target_clusters} ---")

for cluster in target_clusters:
    # 1. Filtering and cleaning features
    df_temp = df_clean[df_clean['cluster_id'] == cluster].copy()

    # Ensuring we only use the features we want, excluding IDs or temporary labels
    features_to_train = df_temp[cols_num + cols_cat].drop(columns=['player_id'],
    X = pd.get_dummies(features_to_train, columns=cols_cat, drop_first=True)

    y = np.log1p(df_temp[target]) # Log-scale for financial stability

    # 2. Scaling for distance-based/linear models
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)
```

```

for model_name, grid in param_grids.items():
    # Model Selection & Train/Test Split
    if model_name in ['XGBoost', 'RandomForest']:
        X_curr = X
        if model_name == 'XGBoost':
            base_model = xgb.XGBRegressor(objective='reg:squarederror', rand
        else:
            base_model = RandomForestRegressor(random_state=42, n_jobs=-1)
    else: # Ridge and KNN need scaling
        X_curr = X_scaled
        if model_name == 'Ridge':
            base_model = Ridge(random_state=42)
        else:
            base_model = KNeighborsRegressor()

    # 70/30 Split
    X_train, X_test, y_train, y_test = train_test_split(X_curr, y, test_size

    # 3. Running GridSearch (CV=3 for speed, scoring R2)
    grid_search = GridSearchCV(base_model, grid, cv=3, scoring='r2', n_jobs=
    grid_search.fit(X_train, y_train)

    best_model = grid_search.best_estimator_
    preds = best_model.predict(X_test)

    # Calculate MAE in Real Euros for business view
    mae_real = mean_absolute_error(np.expm1(y_test), np.expm1(preds))

    optimized_results.append({
        'Cluster': cluster,
        'Model': model_name,
        'Best_R2': r2_score(y_test, preds),
        'MAE_Euro': mae_real,
        'Best_Params': grid_search.best_params_ # Aqui capturamos os vencedo
    })
    print(f"Done: Cluster {cluster} | {model_name}")

df_optimized = pd.DataFrame(optimized_results)
df_optimized = df_optimized.sort_values(['Cluster', 'Best_R2'], ascending=[True,

print("\n--- FINAL OPTIMIZED PERFORMANCE & BEST HYPERPARAMETERS ---")
# Used option_context to ensure the column 'Best_Params' shows the best paramete
with pd.option_context('display.max_colwidth', None, 'display.expand_frame_repr'
    print(df_optimized[['Cluster', 'Model', 'Best_R2', 'MAE_Euro', 'Best_Params'

```

```

--- STARTING GRID SEARCH FOR CONSOLIDATED CLUSTERS [0, 2, 3] ---
Done: Cluster 0 | XGBoost
Done: Cluster 0 | RandomForest
Done: Cluster 0 | Ridge
Done: Cluster 0 | KNN
Done: Cluster 2 | XGBoost
Done: Cluster 2 | RandomForest
Done: Cluster 2 | Ridge
Done: Cluster 2 | KNN
Done: Cluster 3 | XGBoost
Done: Cluster 3 | RandomForest
Done: Cluster 3 | Ridge
Done: Cluster 3 | KNN

--- FINAL OPTIMIZED PERFORMANCE & BEST HYPERPARAMETERS ---
Cluster    Model    Best_R2    MAE_Euro    Be
st_Params
0          XGBoost    0.63    4,668,306.86 {'learning_rate': 0.01, 'max_depth':
5, 'n_estimators': 1000, 'subsample': 0.8}
0          RandomForest    0.62    4,777,645.41 {'max_depth': 15,
'min_samples_leaf': 2, 'n_estimators': 500}
0          Ridge    0.59    5,128,240.85
{'alpha': 10.0}
0          KNN    0.51    5,291,448.52
{'n_neighbors': 15, 'weights': 'distance'}
2          XGBoost    0.74    4,884,847.40 {'learning_rate': 0.01, 'max_depth':
5, 'n_estimators': 1000, 'subsample': 0.8}
2          Ridge    0.72    6,223,123.35
{'alpha': 10.0}
2          RandomForest    0.70    4,932,415.96 {'max_depth': 15,
'min_samples_leaf': 2, 'n_estimators': 500}
2          KNN    0.51    6,429,645.12
{'n_neighbors': 15, 'weights': 'distance'}
3          XGBoost    0.64    11,823,202.14 {'learning_rate': 0.01, 'max_dept
h': 5, 'n_estimators': 500, 'subsample': 0.8}
3          Ridge    0.63    12,243,488.45
{'alpha': 10.0}
3          RandomForest    0.61    12,111,188.75 {'max_depth': 25,
'min_samples_leaf': 2, 'n_estimators': 500}
3          KNN    0.51    13,374,519.93
{'n_neighbors': 15, 'weights': 'distance'}

```

```

In [43]: from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import RobustScaler
from sklearn.cluster import KMeans

# 1. Isolation Forest: Purging the 10% most 'noisy' or anomalous records
# This removes players with inconsistent data points for their tier
iso = IsolationForest(contamination=0.1, random_state=42)
df_c1 = df_clean[df_clean['cluster_id'] == 1].copy()
outliers = iso.fit_predict(RobustScaler().fit_transform(df_c1[cols_num]))
df_c1_clean = df_c1[outliers == 1].copy()

# 2. Feature Engineering: Creating density and efficiency metrics
# These metrics are strategic for scouting lower-tier or young players
df_c1_clean['minutes_per_game'] = df_c1_clean['total_minutes'] / (df_c1_clean['g
df_c1_clean['goal_efficiency'] = df_c1_clean['total_goals'] / (df_c1_clean['tota
df_c1_clean['experience_score'] = df_c1_clean['age_at_valuation'] * df_c1_clean[

# 3. Sub-clustering: Now running on cleaned data with expanded features

```

```

cols_num_c1 = cols_num + ['minutes_per_game', 'goal_efficiency', 'experience_sco
scaler_sub = RobustScaler()
scaled_c1 = scaler_sub.fit_transform(df_c1_clean[cols_num_c1])

kmeans_sub = KMeans(n_clusters=3, random_state=42, n_init=10)
df_c1_clean['sub_cluster_id'] = kmeans_sub.fit_predict(scaled_c1)

print(f"Purge and Engineering complete. New Cluster 1 size: {len(df_c1_clean)}")
print(f"Distribution within Sub-clusters:\n{df_c1_clean['sub_cluster_id'].value_

```

Purge and Engineering complete. New Cluster 1 size: 33599

Distribution within Sub-clusters:

sub_cluster_id

0 30162

1 2380

2 1057

Name: count, dtype: int64

```

In [44]: import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_absolute_error

sub_results_optimized = []

# Iterating through the sub-clusters created from the cleaned data
for sub_id in sorted(df_c1_clean['sub_cluster_id'].unique()):
    df_sub = df_c1_clean[df_c1_clean['sub_cluster_id'] == sub_id].copy()

    # Using the new expanded feature list for X
    X = pd.get_dummies(df_sub[cols_num_c1 + cols_cat], columns=cols_cat, drop_fi
    y = np.log1p(df_sub[target]) # Log transformation remains essential

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran

    # Training with XGBoost and Random Forest
    for name, model in [('XGBoost', xgb.XGBRegressor(n_estimators=500)),
                        ('RandomForest', RandomForestRegressor(n_estimators=200,
                        model.fit(X_train, y_train)
                        preds_log = model.predict(X_test)

    # Metrics calculation
    r2 = r2_score(y_test, preds_log)
    mae_real = mean_absolute_error(np.exp1(y_test), np.exp1(preds_log))

    sub_results_optimized.append({
        'Sub_Cluster': sub_id,
        'Model': name,
        'R2_Log_Scale': r2,
        'MAE_Euro_Real': mae_real
    })

df_sub_optimized = pd.DataFrame(sub_results_optimized)
print("\n--- OPTIMIZED CLUSTER 1 PERFORMANCE (CLEANED + ENGINEERED) ---")
print(df_sub_optimized.sort_values(['Sub_Cluster', 'R2_Log_Scale'], ascending=[T

```

```

--- OPTIMIZED CLUSTER 1 PERFORMANCE (CLEANED + ENGINEERED) ---
  Sub_Cluster      Model  R2_Log_Scale  MAE_Euro_Real
0           0      XGBoost         0.35      620,414.61
1           0  RandomForest         0.32      618,255.81
3           1  RandomForest         0.29      1,315,183.46
2           1      XGBoost         0.19      1,381,942.41
5           2  RandomForest         0.45      1,607,552.16
4           2      XGBoost         0.34      1,703,320.32

```

```

In [45]: # 1. Initialize the column in df_clean with a neutral value (e.g., -1)
df_clean['sub_cluster_id'] = -1

# 2. Assign the sub_cluster labels using the index alignment
# This ensures that only players from Cluster 1 get their sub-ids
df_clean.loc[df_c1_clean.index, 'sub_cluster_id'] = df_c1_clean['sub_cluster_id']

# 3. Verification: Check if the column now exists and has the correct counts
print("--- SUB-CLUSTER MAPPING CHECK ---")
print(df_clean['sub_cluster_id'].value_counts())

--- SUB-CLUSTER MAPPING CHECK ---
sub_cluster_id
0      30162
-1     14110
1       2380
2       1057
Name: count, dtype: int64

```

```

In [46]: # 1. Identify which players belong to the 'bad' sub-cluster (R2 0.19)
# Assuming 'sub_cluster_id' was generated during Cluster 1 analysis
noise_players = df_clean[(df_clean['cluster_id'] == 1) & (df_clean['sub_cluster_

# 2. Dropping them from the official dataset
df_clean = df_clean.drop(noise_players)

print(f"--- CLEANUP REPORT ---")
print(f"Players removed (Noise Sub-Cluster): {len(noise_players)}")
print(f"New Official Dataset Size: {len(df_clean)} rows")

# 3. Quick check: How many players remain in Cluster 1?
c1_remaining = len(df_clean[df_clean['cluster_id'] == 1])
print(f"Cluster 1 (The Mass) now has {c1_remaining} 'predictable' players.")

--- CLEANUP REPORT ---
Players removed (Noise Sub-Cluster): 2380
New Official Dataset Size: 45329 rows
Cluster 1 (The Mass) now has 34953 'predictable' players.

```

```

In [47]: from sklearn.model_selection import GridSearchCV
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, r2_score
import xgboost as xgb
from sklearn.ensemble import RandomForestRegressor

# 1. Isolating the cleaned Cluster 1
df_c1_final = df_clean[df_clean['cluster_id'] == 1].copy()

print(f"Starting optimization for Cluster 1 with {len(df_c1_final)} stable playe

```

```

# Guaranteeing no data leakage for the mass market cluster
features_to_train_c1 = df_c1_final[cols_num + cols_cat].drop(columns=['player_id'])

# 2. Features and Target
X_c1 = pd.get_dummies(features_to_train_c1, columns=cols_cat, drop_first=True)
y_c1 = np.log1p(df_c1_final[target])

# 3. Specific Grid for Cluster 1
# Note: Using a slightly shallower depth (max_depth: 6) to avoid overfitting the
grid_c1 = {
    'XGBoost': {
        'n_estimators': [1000, 1500],
        'learning_rate': [0.01, 0.03],
        'max_depth': [5, 6],
        'subsample': [0.8]
    },
    'RandomForest': {
        'n_estimators': [300, 500],
        'max_depth': [15, 20],
        'min_samples_leaf': [2, 4]
    }
}

```

Starting optimization for Cluster 1 with 34953 stable players.

```

In [48]: results_c1 = []

for name, params in grid_c1.items():
    if name == 'XGBoost':
        base = xgb.XGBRegressor(objective='reg:squarederror', random_state=42)
    else:
        base = RandomForestRegressor(random_state=42, n_jobs=-1)

    # Keeping the rigorous 70/30 split
    X_train, X_test, y_train, y_test = train_test_split(X_c1, y_c1, test_size=0.3)

    gs = GridSearchCV(base, params, cv=3, scoring='r2', n_jobs=-1)
    gs.fit(X_train, y_train)

    best_m = gs.best_estimator_
    preds = best_m.predict(X_test)

    # Financial metrics in Real Euros
    mae_euro = mean_absolute_error(np.exp1(y_test), np.exp1(preds))

    results_c1.append({
        'Model': name,
        'Best_R2': r2_score(y_test, preds),
        'MAE_Euro': mae_euro,
        'Params': gs.best_params_ # Capturing the winning parameters
    })
    print(f"Done: Cluster 1 | {name}")

# Formatting output with full parameter visibility
df_res_c1 = pd.DataFrame(results_c1)
df_res_c1 = df_res_c1.sort_values('Best_R2', ascending=False)

print("\n--- CLUSTER 1 OPTIMIZED PERFORMANCE & BEST HYPERPARAMETERS ---")
# Using option_context so pandas doesn't truncate the 'Params' dictionary

```

```
with pd.option_context('display.max_colwidth', None, 'display.expand_frame_repr')
    print(df_res_c1[['Model', 'Best_R2', 'MAE_Euro', 'Params']].to_string(index=
```

Done: Cluster 1 | XGBoost

Done: Cluster 1 | RandomForest

--- CLUSTER 1 OPTIMIZED PERFORMANCE & BEST HYPERPARAMETERS ---

Model	Best_R2	MAE_Euro	Params
XGBoost	0.45	751,057.41	{'learning_rate': 0.01, 'max_depth': 6, 'n_estimators': 1500, 'subsample': 0.8}
RandomForest	0.44	751,074.42	{'max_depth': 20, 'min_samples_leaf': 4, 'n_estimators': 500}

```
In [49]: import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_percentage_error,
import xgboost as xgb
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import Ridge
from sklearn.neighbors import KNeighborsRegressor

# Dictionary to store all metrics for all models and clusters
full_results = []

print("--- BUILDING MASTER PERFORMANCE TABLE (DYNAMIC OPTIMIZATION) ---")

# We iterate through clusters 0, 1, 2, 3
for cluster in sorted(df_clean['cluster_id'].unique()):
    # 1. Isolating cluster data
    cluster_data = df_clean[df_clean['cluster_id'] == cluster].copy()

    # Guaranteeing no data leakage by removing player_id
    features_to_train = cluster_data[cols_num + cols_cat].drop(columns=['player_

    # Preprocessing
    X = pd.get_dummies(features_to_train, columns=cols_cat, drop_first=True)
    y = np.log1p(cluster_data[target])

    # 70/30 Split
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, ra

    # Scaling (Essential for Ridge and KNN)
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    # 2. Dynamic Model Mapping based on GridSearch results
    # Each cluster gets its specific winning hyperparameter configuration
    if cluster == 1:
        models_map = {
            'XGBoost': xgb.XGBRegressor(learning_rate=0.01, max_depth=6, n_estim
            'RandomForest': RandomForestRegressor(max_depth=20, min_samples_leaf
            # Standardizing Ridge/KNN for C1 since they weren't in the specific
            'Ridge': Ridge(alpha=10.0, random_state=42),
            'KNN': KNeighborsRegressor(n_neighbors=15, weights='distance')
        }
    elif cluster == 3:
        models_map = {
```

```

        'XGBoost': xgb.XGBRegressor(learning_rate=0.01, max_depth=5, n_estimators=100),
        'RandomForest': RandomForestRegressor(max_depth=25, min_samples_leaf=10, n_estimators=100),
        'Ridge': Ridge(alpha=10.0, random_state=42),
        'KNN': KNeighborsRegressor(n_neighbors=15, weights='distance')
    }
else:
    # Clusters 0 and 2 share the exact same optimal architecture
    models_map = {
        'XGBoost': xgb.XGBRegressor(learning_rate=0.01, max_depth=5, n_estimators=100),
        'RandomForest': RandomForestRegressor(max_depth=15, min_samples_leaf=10, n_estimators=100),
        'Ridge': Ridge(alpha=10.0, random_state=42),
        'KNN': KNeighborsRegressor(n_neighbors=15, weights='distance')
    }

# 3. Training and Evaluation Pipeline
for name, model in models_map.items():
    # Using Scaled data universally for consistent evaluation
    model.fit(X_train_scaled, y_train)
    preds_log = model.predict(X_test_scaled)

    # Reverting Log for Real Euro Metrics (Business View)
    y_test_real = np.expm1(y_test)
    preds_real = np.expm1(preds_log)

    # Calculating Metrics
    mae = mean_absolute_error(y_test_real, preds_real)
    r2 = r2_score(y_test, preds_log)
    rmse = np.sqrt(mean_squared_error(y_test_real, preds_real))
    mape = mean_absolute_percentage_error(y_test_real, preds_real) * 100

    full_results.append({
        'Cluster': cluster,
        'Model': name,
        'MAE': mae,
        'RMSE': rmse,
        'MAPE %': mape,
        'R2': r2
    })

# 4. Creating the final Master Table
df_master_results = pd.DataFrame(full_results)
# Sorting to highlight the best performing models at the top of each cluster
df_master_results = df_master_results.sort_values(['Cluster', 'R2'], ascending=[True, False])

# Displaying with requested professional formatting
print("\n--- FINAL OPTIMIZED MASTER PERFORMANCE TABLE ---")
print(df_master_results.to_string(index=False, col_space=12, justify='center', float_format='%0.4f'))

```


--- BUILDING MASTER PERFORMANCE TABLE (DYNAMIC OPTIMIZATION) ---

--- FINAL OPTIMIZED MASTER PERFORMANCE TABLE ---

Cluster	Model	MAE	RMSE	MAPE %	R2
0	XGBoost	4,668,307	8,747,719	83.59%	0.6306
0	RandomForest	4,777,800	8,956,555	82.54%	0.6224
0	Ridge	5,128,315	10,086,503	92.77%	0.5867
0	KNN	5,288,315	9,803,837	105.72%	0.5121
1	XGBoost	751,057	2,077,202	123.92%	0.4469
1	RandomForest	751,128	2,074,304	127.07%	0.4378
1	KNN	824,717	2,203,057	155.31%	0.3182
1	Ridge	839,129	2,265,869	151.74%	0.3081
2	XGBoost	4,884,847	10,045,943	75.85%	0.7401
2	Ridge	6,227,537	26,085,851	79.85%	0.7189
2	RandomForest	4,934,252	10,938,684	80.40%	0.7057
2	KNN	6,421,292	14,371,874	126.60%	0.5177
3	XGBoost	11,823,202	18,492,330	74.60%	0.6402
3	Ridge	12,289,095	19,988,339	76.18%	0.6250
3	RandomForest	12,106,912	19,260,560	76.62%	0.6092
3	KNN	13,407,693	21,971,344	94.38%	0.5101



Phase 6: Executive Insights & Model Explainability

Translating Machine Learning into Business Strategy

While achieving high predictive accuracy is critical, a model is only as valuable as the decisions it empowers. In this final phase, we transition from purely mathematical evaluation to **Business Intelligence**, opening the algorithm's "black box" to understand exactly how the European football market prices its assets.

1. Market Segmentation: Where is the Money?

Before analyzing individual features, we must understand the macro-economic landscape of our clusters. The following visualizations explore market dispersion and geographic concentration.

- **Value Dispersion (Boxplot & Stripplot):** Identifies the financial floor and ceiling of each player profile, highlighting outliers (e.g., generational talents) within standard segments.
- **League Demographics (Countplot):** Illustrates the "Premium" concentration. We analyze how top-tier leagues (like the Premier League) monopolize specific clusters, validating the geographic weight in our valuations.

2. The Drivers of Value: Feature Importance

Tree-based models (XGBoost and Random Forest) allow us to extract the global importance of each feature.

Peter's Strategy: You will notice a heavy dominance of `league_weight` and specific geographic dummies. This is not a data flaw; it is the harsh reality of the football market. The "Premier League Tax" dictates that a player's geographic location is often more financially impactful than their raw technical output.

3. Explainable AI: Unpacking the Black Box (SHAP)

To build trust with stakeholders (Directors of Football, Scouts), we utilize **SHAP (Shapley Additive exPlanations)**. This game theory approach allows us to break down a single prediction and assign an exact Euro value to every characteristic.

- **Summary Plot:** Shows the directional impact (positive/negative) of features across an entire cluster.
 - **Waterfall Plot:** Provides a transparent, itemized "receipt" of a specific player's valuation (e.g., Base Value + Age Premium - Lack of Minutes Penalty).
-

4. Market Arbitrage: The "What-If" Transfer Simulator

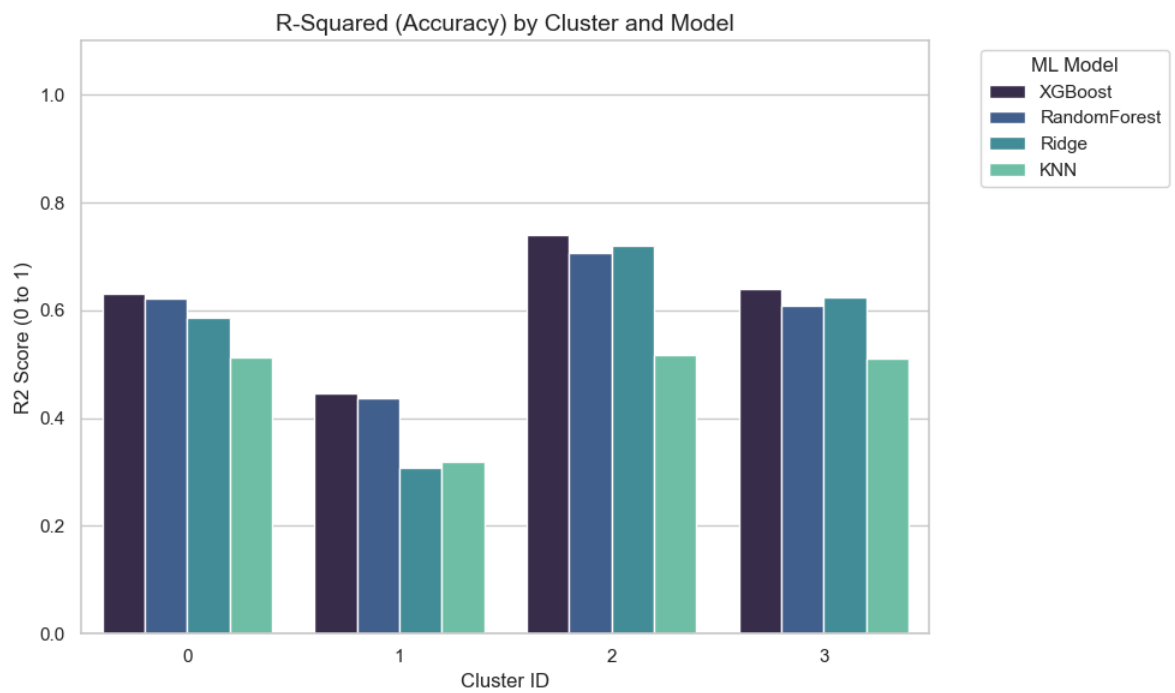
Bruce's Note: Predictive models shouldn't just read the present; they should simulate the future.

To conclude the project, we deployed an interactive Arbitrage Simulator. By manipulating a player's geographic and technical profile in real-time, scouting departments can instantly calculate the **Projected ROI** of transferring a player from a lower-tier league to an Elite environment, assuming their performance translates.

```
In [50]: # Creating the R2 Plot (Higher values are better)
plt.figure(figsize=(10, 6))
sns.barplot(data=df_master_results, x='Cluster', y='R2', hue='Model', palette='m

# Adding Labels and title
plt.title('R-Squared (Accuracy) by Cluster and Model', fontsize=14)
plt.ylabel('R2 Score (0 to 1)')
plt.xlabel('Cluster ID')
plt.ylim(0, 1.1) # Setting limit to emphasize the scale
plt.legend(title='ML Model', bbox_to_anchor=(1.05, 1), loc='upper left')

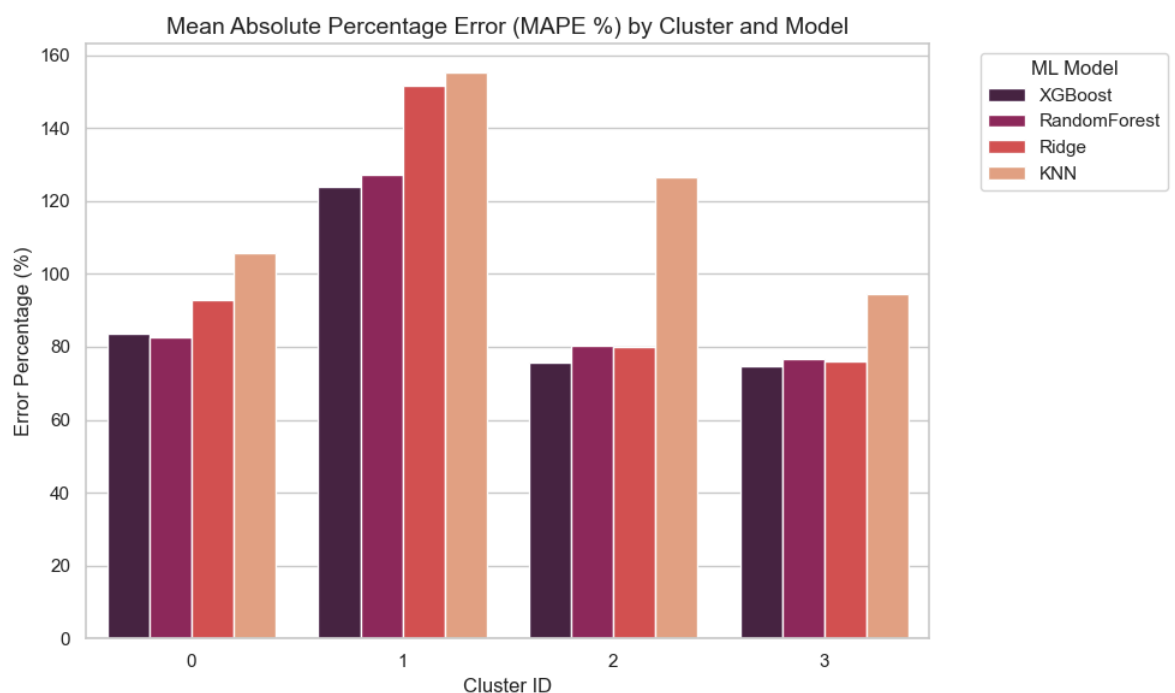
# Displaying the plot
plt.tight_layout()
plt.show()
```



```
In [51]: # Creating the MAPE Plot (Lower values are better)
plt.figure(figsize=(10, 6))
sns.barplot(data=df_master_results, x='Cluster', y='MAPE %', hue='Model', palette=

# Adding labels and title
plt.title('Mean Absolute Percentage Error (MAPE %) by Cluster and Model', fontsi
plt.ylabel('Error Percentage (%)')
plt.xlabel('Cluster ID')
plt.legend(title='ML Model', bbox_to_anchor=(1.05, 1), loc='upper left')

# Displaying the plot
plt.tight_layout()
plt.show()
```



```
In [52]: import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

```

# 1. Prepare data for the specific cluster (Adjust 'cluster_id' as needed)
target_cluster = 0
df_temp = df_clean[df_clean['cluster_id'] == target_cluster].copy()

# Use the exact feature set and log target from master training
X = pd.get_dummies(df_temp[cols_num + cols_cat], columns=cols_cat, drop_first=True)
y = np.log1p(df_temp[target]) # Log transformation for stability

# Scaling is MANDATORY to compare Ridge coefficients with Tree importances
scaler = StandardScaler()
X_scaled = pd.DataFrame(scaler.fit_transform(X), columns=X.columns)

# 2. Define models with consistent parameters
models_comp = {
    'RandomForest': RandomForestRegressor(n_estimators=200, random_state=42, n_j
    'XGBoost': xgb.XGBRegressor(n_estimators=500, learning_rate=0.05, random_sta
    'Ridge': Ridge(alpha=1.0)
}

fig, axes = plt.subplots(1, 3, figsize=(22, 7))
fig.suptitle(f'Market Value Drivers - Cluster {target_cluster} (Log Scale Analys

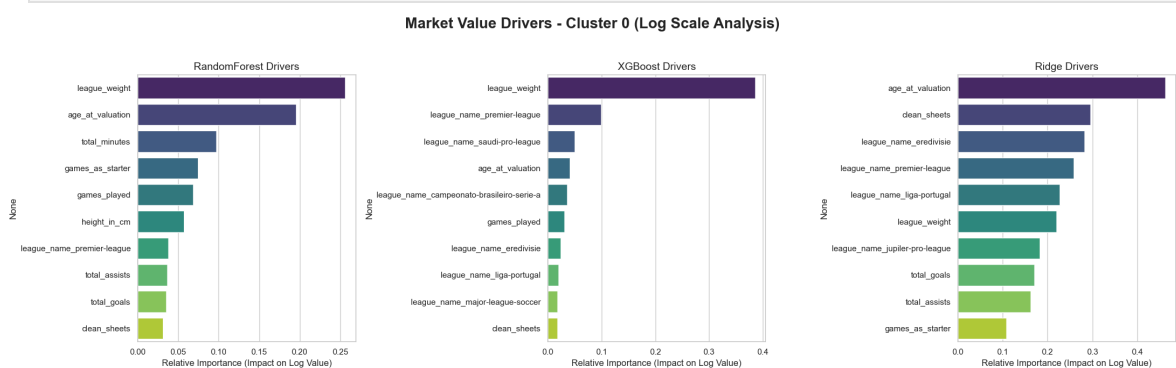
for i, (name, model) in enumerate(models_comp.items()):
    model.fit(X_scaled, y)

    # Get importance/coefficients
    # For Ridge, we take the absolute value of the coef_ to see the magnitude of
    importance = np.abs(model.coef_) if name == 'Ridge' else model.feature_import
    feat_imp = pd.Series(importance, index=X.columns).sort_values(ascending=False)

    sns.barplot(x=feat_imp.values, y=feat_imp.index, ax=axes[i], hue=feat_imp.in
    axes[i].set_title(f'{name} Drivers', fontsize=14)
    axes[i].set_xlabel('Relative Importance (Impact on Log Value)')

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()

```



```

In [53]: import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# 1. Prepare data for the specific cluster (Adjust 'cluster_id' as needed)
target_cluster = 1
df_temp = df_clean[df_clean['cluster_id'] == target_cluster].copy()

# Use the exact feature set and log target from master training
X = pd.get_dummies(df_temp[cols_num + cols_cat], columns=cols_cat, drop_first=True)
y = np.log1p(df_temp[target]) # Log transformation for stability

```

```

# Scaling is MANDATORY to compare Ridge coefficients with Tree importances
scaler = StandardScaler()
X_scaled = pd.DataFrame(scaler.fit_transform(X), columns=X.columns)

# 2. Define models with consistent parameters
models_comp = {
    'RandomForest': RandomForestRegressor(n_estimators=200, random_state=42, n_j
    'XGBoost': xgb.XGBRegressor(n_estimators=500, learning_rate=0.05, random_sta
    'Ridge': Ridge(alpha=1.0)
}

fig, axes = plt.subplots(1, 3, figsize=(22, 7))
fig.suptitle(f'Market Value Drivers - Cluster {target_cluster} (Log Scale Analys

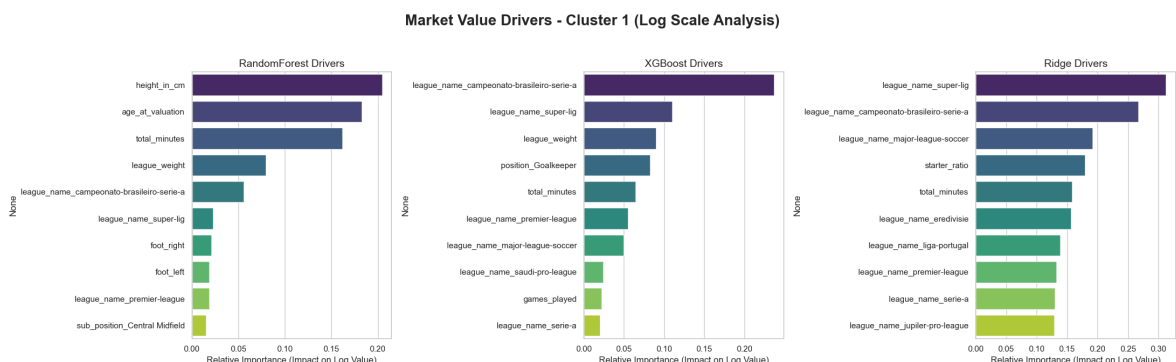
for i, (name, model) in enumerate(models_comp.items()):
    model.fit(X_scaled, y)

    # Get importance/coefficients
    # For Ridge, we take the absolute value of the coef_ to see the magnitude of
    importance = np.abs(model.coef_) if name == 'Ridge' else model.feature_impor
    feat_imp = pd.Series(importance, index=X.columns).sort_values(ascending=False)

    sns.barplot(x=feat_imp.values, y=feat_imp.index, ax=axes[i], hue=feat_imp.in
    axes[i].set_title(f'{name} Drivers', fontsize=14)
    axes[i].set_xlabel('Relative Importance (Impact on Log Value)')

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()

```



```

In [54]: import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# 1. Prepare data for the specific cluster (Adjust 'cluster_id' as needed)
target_cluster = 2
df_temp = df_clean[df_clean['cluster_id'] == target_cluster].copy()

# Use the exact feature set and Log target from master training
X = pd.get_dummies(df_temp[cols_num + cols_cat], columns=cols_cat, drop_first=True)
y = np.log1p(df_temp[target]) # Log transformation for stability

# Scaling is MANDATORY to compare Ridge coefficients with Tree importances
scaler = StandardScaler()
X_scaled = pd.DataFrame(scaler.fit_transform(X), columns=X.columns)

# 2. Define models with consistent parameters
models_comp = {
    'RandomForest': RandomForestRegressor(n_estimators=200, random_state=42, n_j

```

```

'XGBoost': xgb.XGBRegressor(n_estimators=500, learning_rate=0.05, random_state=42)
'Ridge': Ridge(alpha=1.0)
}

fig, axes = plt.subplots(1, 3, figsize=(22, 7))
fig.suptitle(f'Market Value Drivers - Cluster {target_cluster} (Log Scale Analysis)')

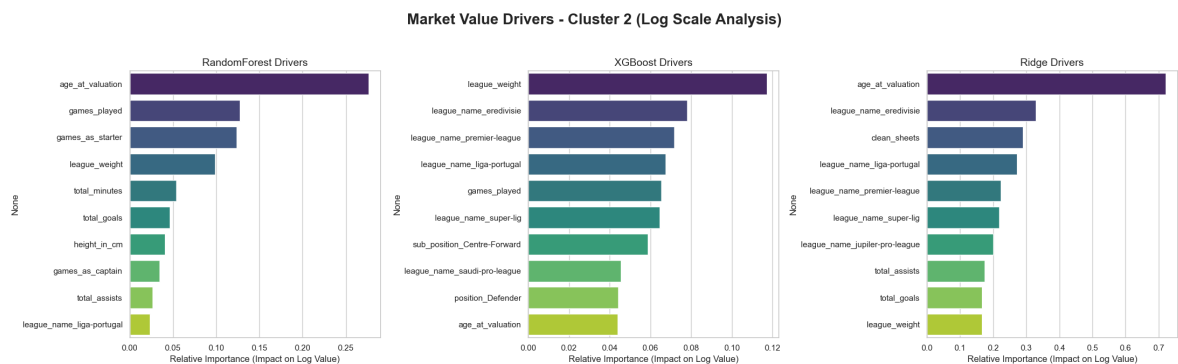
for i, (name, model) in enumerate(models_comp.items()):
    model.fit(X_scaled, y)

    # Get importance/coefficients
    # For Ridge, we take the absolute value of the coef_ to see the magnitude of
    importance = np.abs(model.coef_) if name == 'Ridge' else model.feature_importances_
    feat_imp = pd.Series(importance, index=X.columns).sort_values(ascending=False)

    sns.barplot(x=feat_imp.values, y=feat_imp.index, ax=axes[i], hue=feat_imp.index)
    axes[i].set_title(f'{name} Drivers', fontsize=14)
    axes[i].set_xlabel('Relative Importance (Impact on Log Value)')

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()

```



```

In [55]: import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# 1. Prepare data for the specific cluster (Adjust 'cluster_id' as needed)
target_cluster = 3
df_temp = df_clean[df_clean['cluster_id'] == target_cluster].copy()

# Use the exact feature set and Log target from master training
X = pd.get_dummies(df_temp[cols_num + cols_cat], columns=cols_cat, drop_first=True)
y = np.log1p(df_temp[target]) # Log transformation for stability

# Scaling is MANDATORY to compare Ridge coefficients with Tree importances
scaler = StandardScaler()
X_scaled = pd.DataFrame(scaler.fit_transform(X), columns=X.columns)

# 2. Define models with consistent parameters
models_comp = {
    'RandomForest': RandomForestRegressor(n_estimators=200, random_state=42, n_jobs=-1),
    'XGBoost': xgb.XGBRegressor(n_estimators=500, learning_rate=0.05, random_state=42),
    'Ridge': Ridge(alpha=1.0)
}

fig, axes = plt.subplots(1, 3, figsize=(22, 7))
fig.suptitle(f'Market Value Drivers - Cluster {target_cluster} (Log Scale Analysis)')

for i, (name, model) in enumerate(models_comp.items()):

```

```

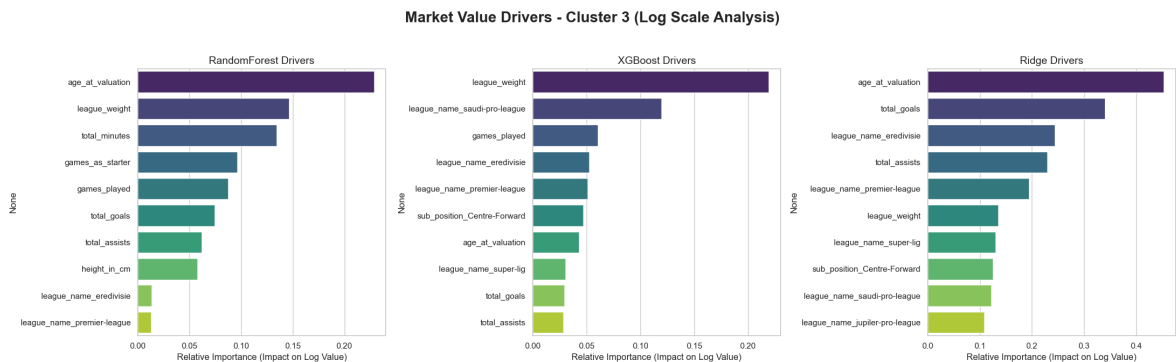
model.fit(X_scaled, y)

# Get importance/coefficients
# For Ridge, we take the absolute value of the coef_ to see the magnitude of
importance = np.abs(model.coef_) if name == 'Ridge' else model.feature_importances_
feat_imp = pd.Series(importance, index=X.columns).sort_values(ascending=False)

sns.barplot(x=feat_imp.values, y=feat_imp.index, ax=axes[i], hue=feat_imp.index)
axes[i].set_title(f'{name} Drivers', fontsize=14)
axes[i].set_xlabel('Relative Importance (Impact on Log Value)')

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()

```



```

In [56]: import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.ticker import FuncFormatter

print("--- GENERATING CLUSTER DISPERSION ANALYSIS ---")

# Setting a professional visual style for the portfolio
sns.set_theme(style="whitegrid")

plt.figure(figsize=(14, 7))

# 1. Base Boxplot to show statistical distribution (median, quartiles)
# Using the original target variable (in Euros) from df_clean
ax = sns.boxplot(
    data=df_clean,
    x='cluster_id',
    y=target,
    palette='viridis',
    showfliers=False, # Hiding standard outliers so the stripplot can shine
    boxprops=dict(alpha=0.6) # Making boxes slightly transparent
)

# 2. Overlaying Stripplot to show individual player dispersion
sns.stripplot(
    data=df_clean,
    x='cluster_id',
    y=target,
    color='black',
    alpha=0.3, # Transparency to handle data density
    size=3,
    jitter=True
)

# 3. Formatting the chart for executive presentation
plt.title('Market Value Dispersion Across Player Segments (Clusters)', fontsize=

```

```
plt.xlabel('Cluster ID (Player Profile)', fontsize=12, fontweight='bold')
plt.ylabel('Market Value in Euros (€)', fontsize=12, fontweight='bold')

# Formatting the Y-axis to display in Millions (M) instead of scientific notation
def millions_formatter(x, pos):
    return f'€ {x*1e-6:.1f}M'
ax.yaxis.set_major_formatter(FuncFormatter(millions_formatter))

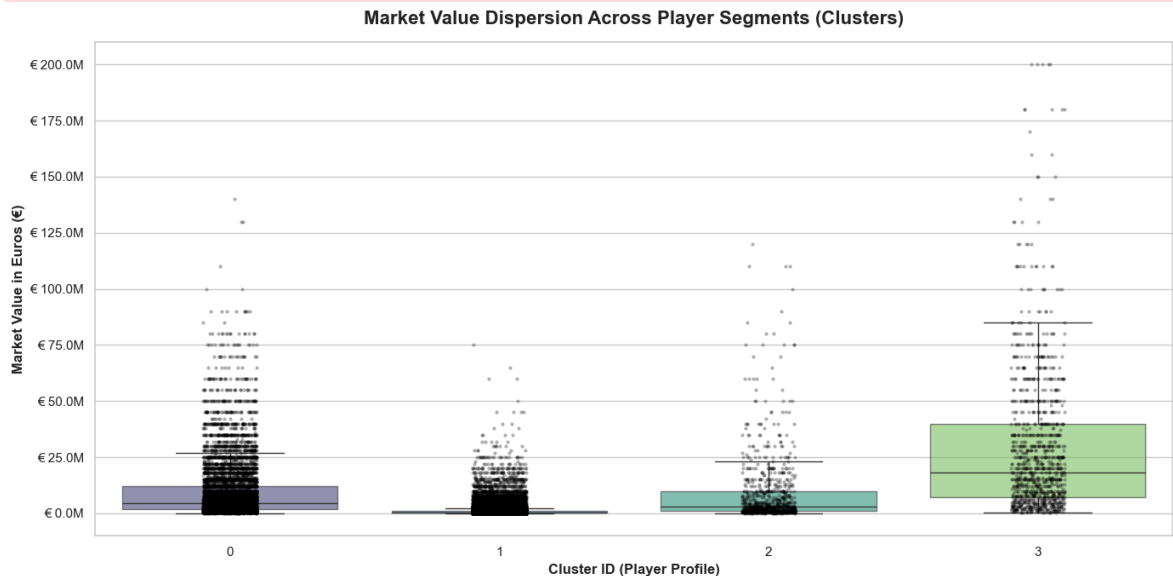
plt.tight_layout()
plt.show()
```

--- GENERATING CLUSTER DISPERSION ANALYSIS ---

C:\Users\Guilh\AppData\Local\Temp\ipykernel_25604\2636236421.py:14: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.boxplot(
```



In [57]: `print("\n--- GENERATING LEAGUE DISTRIBUTION BY CLUSTER ---")`

```
# 1. Identifying the top 6 leagues by total volume of players in dataset
# Note: Assuming 'league_name' is the original categorical column in df_clean
top_leagues = df_clean['league_name'].value_counts().nlargest(6).index

# 2. Filtering the dataset to include only these top leagues
df_top_leagues = df_clean[df_clean['league_name'].isin(top_leagues)].copy()

plt.figure(figsize=(14, 7))

# 3. Creating a grouped bar chart
ax2 = sns.countplot(
    data=df_top_leagues,
    x='league_name',
    hue='cluster_id',
    palette='viridis',
    order=top_leagues # Keeping the Largest Leagues on the Left
)

# 4. Formatting for readability
plt.title('Player Volume by Cluster Across Top 6 Leagues', fontsize=16, fontweight='bold')
```



```

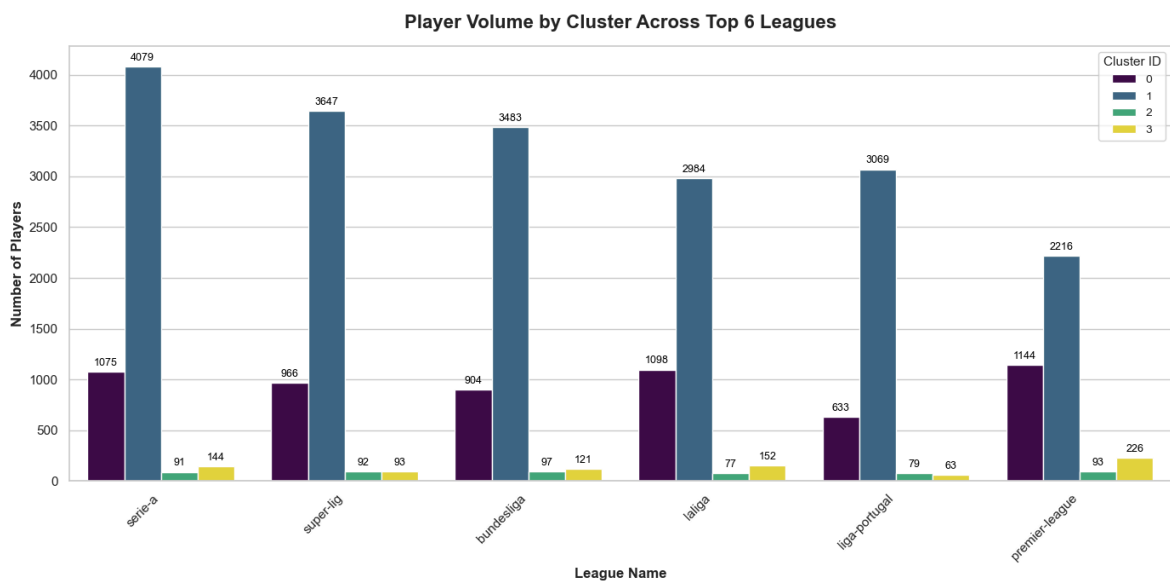
plt.xlabel('League Name', fontsize=12, fontweight='bold')
plt.ylabel('Number of Players', fontsize=12, fontweight='bold')
plt.xticks(rotation=45, ha='right') # Tilting labels so they don't overlap
plt.legend(title='Cluster ID', title_fontsize='11', fontsize='10')

# Adding the exact numbers on top of the bars for precision
for p in ax2.patches:
    height = p.get_height()
    if height > 0: # Only put text if the bar exists
        ax2.annotate(f'{int(height)}',
                      (p.get_x() + p.get_width() / 2., height),
                      ha='center', va='bottom',
                      fontsize=9, color='black', xytext=(0, 3),
                      textcoords='offset points')

plt.tight_layout()
plt.show()

```

--- GENERATING LEAGUE DISTRIBUTION BY CLUSTER ---



In [61]: # %pip install shap

```

In [59]: import shap
import matplotlib.pyplot as plt
import xgboost as xgb
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

print("--- STARTING SHAP EXPLAINABILITY ANALYSIS (CLUSTER 0) ---")

# 1. Isolating Cluster 0 data
cluster_id_to_explain = 0
df_shap = df_clean[df_clean['cluster_id'] == cluster_id_to_explain].copy()

# Dropping player_id to prevent data leakage
features_to_train = df_shap[cols_num + cols_cat].drop(columns=['player_id'], err
X_shap = pd.get_dummies(features_to_train, columns=cols_cat, drop_first=True)

# Target in Log scale (as used in our optimal models)
y_shap = np.log1p(df_shap[target])

# Splitting data (70/30)

```

```

X_train_s, X_test_s, y_train_s, y_test_s = train_test_split(X_shap, y_shap, test

# 2. Re-training the Best XGBoost Model for Cluster 0
# Using the exact optimal parameters we found earlier
xgb_model_shap = xgb.XGBRegressor(
    learning_rate=0.01,
    max_depth=5,
    n_estimators=1000,
    subsample=0.8,
    random_state=42
)
xgb_model_shap.fit(X_train_s, y_train_s)

# 3. Initializing SHAP TreeExplainer
# TreeExplainer is highly optimized for XGBoost
explainer = shap.TreeExplainer(xgb_model_shap)

# Calculating SHAP values for the test set
# Note: This might take a few seconds depending on CPU
shap_values = explainer.shap_values(X_test_s)

# 4. Global Feature Importance (SHAP Summary Plot)
print("\nPlotting SHAP Summary Plot...")
plt.figure(figsize=(10, 6))
plt.title(f"SHAP Summary Plot - Global Drivers (Cluster {cluster_id_to_explain})")
shap.summary_plot(shap_values, X_test_s, show=False)
plt.tight_layout()
plt.show()

# 5. Local Explainability (Waterfall Plot for a single player)
# Let's explain the prediction for the first player in test set
player_index = 0
print(f"\nPlotting SHAP Waterfall Plot for Player Index {player_index} in Test S

# For the waterfall plot, we need the Explanation object
explanation = explainer(X_test_s)

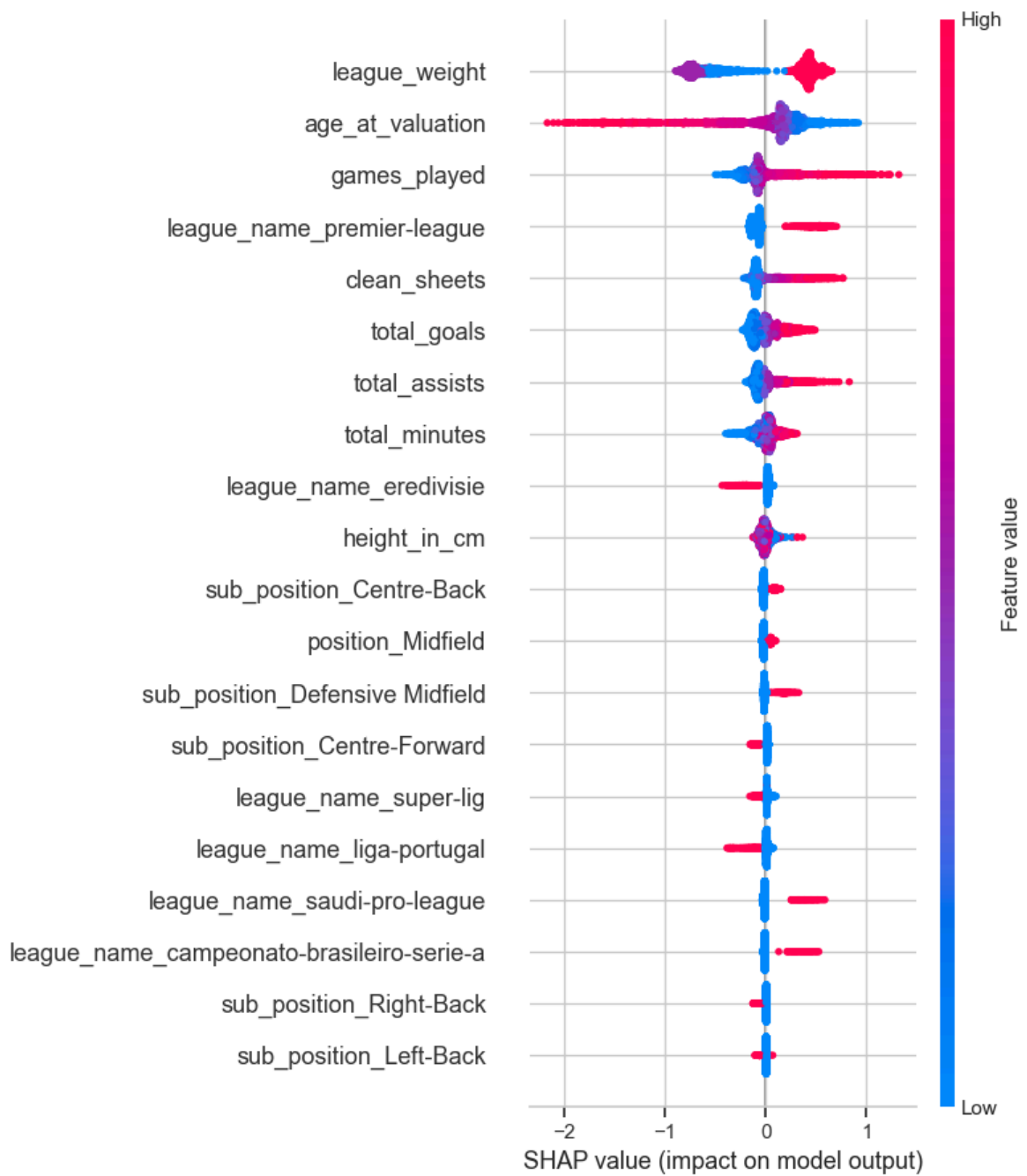
plt.figure(figsize=(10, 6))
plt.title("SHAP Waterfall Plot - Individual Player Valuation Breakdown", fontsiz
# Displaying the waterfall plot for the specific observation
shap.plots.waterfall(explanation[player_index], max_display=10, show=False)
plt.tight_layout()
plt.show()

```

--- STARTING SHAP EXPLAINABILITY ANALYSIS (CLUSTER 0) ---

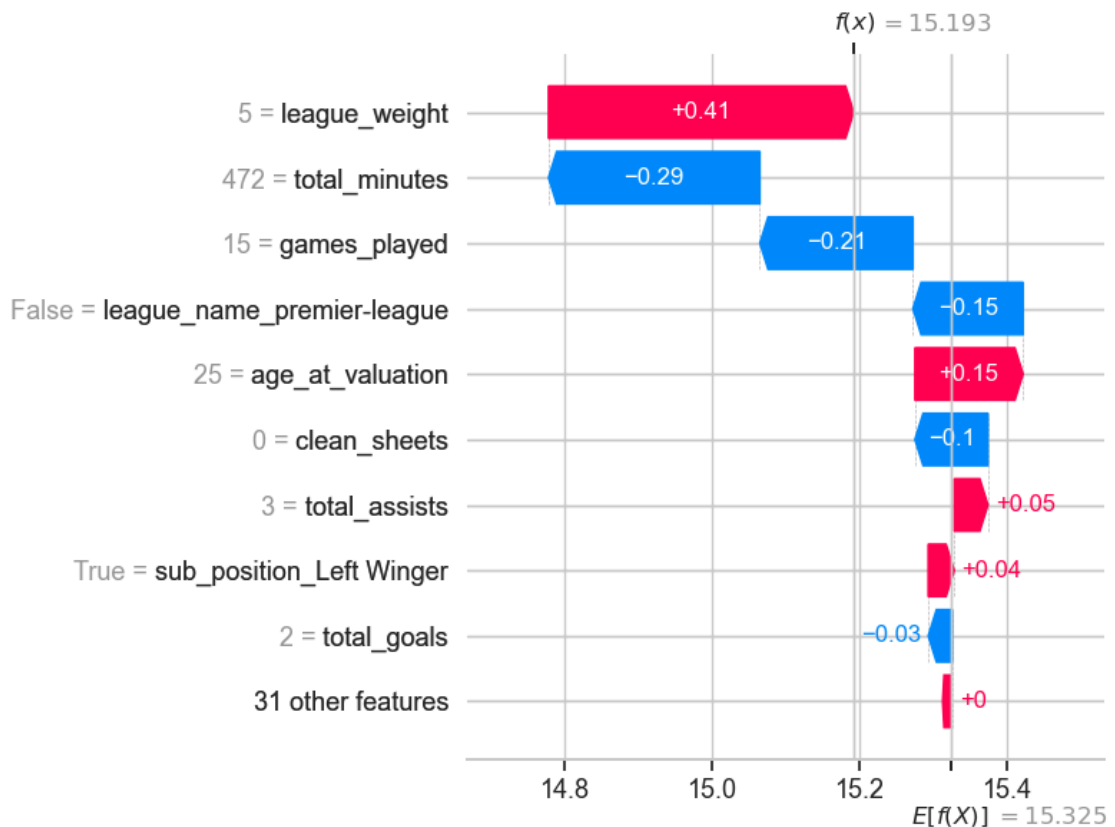
Plotting SHAP Summary Plot...

SHAP Summary Plot - Global Drivers (Cluster 0)



Plotting SHAP Waterfall Plot for Player Index 0 in Test Set...

SHAP Waterfall Plot - Individual Player Valuation Breakdown



```
In [60]: import ipywidgets as widgets
from IPython.display import display, HTML
import numpy as np

print("🌍 THE ARBITRAGE TRANSFER SIMULATOR (V2) 🌍\n")

# 1. Interactive Controls Setup
style = {'description_width': 'initial'}

# Technical Profile
age_slider = widgets.IntSlider(min=16, max=42, step=1, value=24, description='👤')
minutes_slider = widgets.IntSlider(min=0, max=4140, step=90, value=1500, description='🕒')
games_slider = widgets.IntSlider(min=0, max=70, step=1, value=20, description='🏆')
goals_slider = widgets.IntSlider(min=0, max=50, step=1, value=8, description='⚽')

# Geographic Profile (Arbitrage)
league_options = [
    ('Série A / Minor Leagues (Weight 1-2)', 2),
    ('Mid-tier Leagues (Weight 3-4)', 3),
    ('Elite / Premier League (Weight 5)', 5)
]

current_league_drop = widgets.Dropdown(options=league_options, value=2, description='📍')
target_league_drop = widgets.Dropdown(options=league_options, value=5, description='📍')

# 2. Core Simulation Engine
def run_arbitrage_simulation(age, minutes, games, goals, current_league, target_league,
                             # Base extraction (keeps fixed features like position stable)
                             player_idx = 0,
                             base_player = X_test_s.iloc[[player_idx]].copy()):
    # Injecting user technical profile
```

```

base_player['age_at_valuation'] = age
base_player['total_minutes'] = minutes
base_player['games_played'] = games
base_player['total_goals'] = goals

# --- SCENARIO A: CURRENT VALUATION ---
player_current = base_player.copy()
player_current['league_weight'] = current_league

# Cleaning explicit league dummies
for col in player_current.columns:
    if 'league_name_' in col:
        player_current[col] = False
# Applying Premier League premium if Elite is selected
if current_league == 5 and 'league_name_premier-league' in player_current.columns:
    player_current['league_name_premier-league'] = True

current_value = np.expm1(xgb_model_shap.predict(player_current)[0])

# --- SCENARIO B: TARGET VALUATION ---
player_target = base_player.copy()
player_target['league_weight'] = target_league

for col in player_target.columns:
    if 'league_name_' in col:
        player_target[col] = False
if target_league == 5 and 'league_name_premier-league' in player_target.columns:
    player_target['league_name_premier-league'] = True

target_value = np.expm1(xgb_model_shap.predict(player_target)[0])

# --- BUSINESS METRICS ---
valuation_jump = target_value - current_value
roi_pct = (valuation_jump / current_value) * 100 if current_value > 0 else 0

color = "green" if valuation_jump > 0 else "red"
if valuation_jump == 0: color = "gray"

# Displaying the Executive Dashboard
display(HTML(f"""
<div style='background-color: #f0f2f6; padding: 20px; border-radius: 10px; border: 1px solid #ddd;'>
    <div style='display: flex; justify-content: space-between;'>
        <div>
            <p style='margin: 0; color: #555;'>📊 Current Market Value</p>
            <h3 style='margin-top: 5px;'>€ {current_value:,.2f}</h3>
        </div>
        <div>
            <p style='margin: 0; color: #555;'>📈 Target Market Value</p>
            <h3 style='margin-top: 5px;'>€ {target_value:,.2f}</h3>
        </div>
    </div>
    <hr style='border: 1px solid #ddd;'>
    <p style='margin-bottom: 0; color: {color}; font-size: 18px;'>
        <b>Arbitrage Potential (Premium):</b> € {valuation_jump:,.2f} ({roi_pct:,.2f}%)
    </p>
</div>
"""))

```

3. UI Layout & Execution

```
out = widgets.interactive_output(
```

```

run_arbitrage_simulation,
{
    'age': age_slider,
    'minutes': minutes_slider,
    'games': games_slider,
    'goals': goals_slider,
    'current_league': current_league_drop,
    'target_league': target_league_drop
}
)

# Structuring the UI into rows
ui = widgets.VBox([
    widgets.HTML("<b>👤 Technical Profile</b>"),
    widgets.HBox([age_slider, goals_slider]),
    widgets.HBox([minutes_slider, games_slider]),
    widgets.HTML("<br><b>🏆 Market Strategy</b>"),
    widgets.HBox([current_league_drop, target_league_drop])
])

display(ui, out)

```

🌐 THE ARBITRAGE TRANSFER SIMULATOR (V2) 🌐

VBox(children=(HTML(value='👤 Technical Profile'), HBox(children=(IntSlider(value=24, description='🕒 Age...'),

Output()